



ZXUT 系列多通道超声波检测系统

用户使用指南

本文档适用于：

- **ZXUT-NET**，基于千兆以太网接口
- **ZXUT-PCI**，基于 PCI 总线接口

武汉泽旭科技有限公司

版本：V2.2 修订：2019 年 10 月

© 2007-2019 年 武汉泽旭科技有限公司 版权所有

保留本软件开发指南的最终解释权。

特别说明

为了提升仪器可靠性、改进设计与性能，**武汉泽旭科技有限公司**保留对本使用说明书进行修改的权利，若有更改恕不另行通知。

本使用说明书包含受版权法保护的专属信息，保留所有权利。本使用说明书的任何部分，若事先未经**武汉泽旭科技有限公司**书面许可，不得以机械、电子或其它任何形式或手段进行复制与传播。

武汉泽旭科技有限公司对因使用本使用说明书中所包含的信息而导致直接或间接的损害一概不予负责。

商标声明



为**武汉泽旭科技有限公司**的注册商标。本使用说明书中可能提及其它的商标和商标名称，来代表注册该商标和名称的实体或其产品。这些商标为各自公司注册所有。**武汉泽旭科技有限公司**并不拥有这些商标的所有权。

联系我们

若阁下在使用本仪器时有任何的建议、意见或疑问，请按以下方式联系我们。

通讯地址：武汉市武昌区民主路 717 号金都华庭 A 座 1605 室 **邮编：**430063

联系人：左林 **手机：**13986089722 **电子邮件：**embedbird@126.com

电话：027-88236080 **传真：**027-88210023

修订历史记录

版本	日期	修订人	描述
1.0	2019.02.28	左林	。初始版本
2.0	2019.04.01	左林	。重新修订数据结构，简化设备初始化
2.1	2019.05.23	左林	。加入最大重复频率为 50KHz 的限制； 。加入通道时间占额的设置； 。加入同步延迟的设置。
2.2	2019.10.08	左林	。首次融合 ZXUT-NET 和 ZXUT-PCI 的用户使用指南； 。加入板卡硬件部分的描述； 。加入 IO 板的功能描述； 。提供 32 位和 64 位两个版本。

目录

目录	5
第 1 章 硬件描述及技术指标	11
1.1 产品概述	11
1.2 主要技术指标	11
1.3 PCI 板卡硬件构成	12
1.3.1 超声波探头插座	12
1.3.2 超声波发射&接收模块	13
1.3.3 模数转换	13
1.3.4 高压模块	13
1.3.5 PCI 桥接控制器	13
1.3.6 现场可编程门阵列（FPGA）	13
1.3.7 温度监控	13
1.3.8 DI、DO 和 ENC	13
1.3.9 同步输入与输出	14
1.4 PCI 板卡硬件的安装	15
第 2 章 软件开发概述	17
2.1 内容描述	17
2.2 软件分层模型	17
2.3 演示软件	17
2.3.1 探伤参数文件	18
2.4 中间件	18
2.5 ZXUT-NET 软件编程	19
2.5.1 客户机软件	19
2.5.2 服务器软件	19
2.6 ZXUT-PCI 驱动程序及安装	19
2.7 Windows 64 位操作系统的特别考虑	23
2.7.1 运行命令行	23
2.7.2 设置操作系统为测试模式	23
2.7.3 重启操作系统	23
2.8 常用缩略语	24
第 3 章 中间件（DLL）接口命令详述	27
3.1 包含头文件	27
3.2 ZxutCmn.h 包含内容简述	27
3.2.1 常用数据类型定义	27

3.2.2 版本号	28
3.3 ZxutExp.h 包含内容简述	28
3.3.1 设备制造商专用 API 函数	28
3.3.2 API 函数返回码	28
3.4 设备接口函数	28
3.4.1 打开探伤设备 (OpenDevice)	29
3.4.2 关闭探伤设备 (CloseDevice)	30
3.4.3 设置网络配置 (SetNetCfg)	30
3.4.4 获取探伤通道板序列号 (GetBrdSn)	31
3.4.5 设置核心参数指针 (SetCoreParamPtr)	31
3.4.6 设置最大采集率 (SetMaxDaqRate)	32
3.5 系统参数接口函数	32
3.5.1 通用系统参数接口函数	32
3.5.1.1 设置采样使能 (SetSmpEnb)	32
3.5.1.2 设置发射电压 (SetTransVol)	32
3.5.1.3 设置触发模式 (SetTrigMode)	33
3.5.1.4 设置触发极性 (SetTrigPol)	33
3.5.1.5 设置扫查编码器 (SetScanEnc)	34
3.5.1.6 设置扫查间隔 (SetEncSpan)	34
3.5.1.7 设置板间同步模式 (SetBrdSync)	35
3.5.1.8 设置重复频率 (SetRptFreq)	35
3.5.2 编码器 (ENC) 参数接口	36
3.5.2.1 设置编码器滤波 (SetEncFltr)	36
3.5.2.2 设置编码器 AB 相计数器 (SetEabCntr)	37
3.5.2.3 设置编码器 Z 相计数器 (SetEzCntr)	37
3.5.2.4 设置编码器使能 (SetEncEnb)	37
3.5.2.5 设置编码器类型 (SetEncType)	37
3.5.2.6 设置编码器极性 (SetEncPol)	38
3.5.2.7 设置编码器回零使能 (SetEncZe)	38
3.5.3 数字输入 (DI) 参数接口	39
3.5.3.1 设置数字输入使能 (SetDiEnb)	39
3.5.3.2 设置数字输入模式 (SetDiMode)	39
3.5.3.3 设置数字输入极性 (SetDiPol)	40
3.5.3.4 设置数字输入清除 (SetDiClr)	40
3.5.3.5 设置数字输入宽度 (SetDiWidth)	41
3.5.3.6 设置数字输入延迟 (SetDiDly)	41
3.5.3.7 设置数字输入时基 (SetDiBase)	41
3.5.4 数字输出 (DO) 参数接口	42
3.5.4.1 设置数字输出使能 (SetDoEnb)	42
3.5.4.2 设置数字输出模式 (SetDoMode)	42
3.5.4.3 设置数字输出极性 (SetDoPol)	43
3.5.4.4 设置数字输出命令 (SetDoCmd)	43
3.5.4.5 设置数字输出宽度 (SetDoWidth)	44
3.5.4.6 设置数字输出延迟 (SetDoDly)	44

3.5.4.7 设置数字输出时基 (SetDoBase)	44
3.6 通道参数接口函数	45
3.6.1 通用通道参数接口	45
3.6.1.1 设置通道使能 (SetChanEnb)	45
3.6.1.2 设置收发模式 (SetRtMode)	46
3.6.1.3 设置数字滤波器 (SetDigFltr)	46
3.6.1.4 设置模拟滤波器 (SetAngFltr)	47
3.6.1.5 设置检波模式 (SetDemMode)	47
3.6.1.6 设置增益值 (SetdBNum)	48
3.6.1.7 设置发射脉冲宽度 (SetPulWidth)	48
3.6.1.8 设置 FFT 闸门 (SetFftGate)	49
3.6.1.9 设置回波模式 (SetEchoMode)	49
3.6.1.10 设置采样深度 (SetSmplDepth)	50
3.6.1.11 设置零位偏移 (SetOrigDly)	51
3.6.1.12 设置分频比 (SetFreqRatio)	51
3.6.1.13 设置平均次数 (SetAvgTimes)	52
3.6.1.14 设置数据开关 (SetDataSwitch)	52
3.6.2 TCG 通道参数接口	53
3.6.2.1 设置 TCG 使能 (SetTcgEnb)	53
3.6.2.2 设置 TCG 总时间 (SetTcgTotTime)	53
3.6.2.3 设置 TCG 测试点点数 (SetTcgTpTot)	53
3.6.2.4 设置 TCG 测试点索引 (SetTcgTpIdx)	54
3.6.2.5 设置 TCG 测试点时间 (SetTcgTpTime)	54
3.6.2.6 设置 TCG 测试点增益 (SetTcgTpGain)	54
3.6.3 闸门参数接口	54
3.6.3.1 设置闸门起点 (SetGateStart)	55
3.6.3.2 设置闸门宽度 (SetGateWidth)	55
3.6.3.3 设置闸门高度 (SetGateHeight)	56
3.6.3.4 设置闸门极性 (SetGatePol)	56
3.6.4 界面波闸门跟踪	57
3.6.4.1 设置跟踪使能 (SetTraceEnb)	57
3.6.4.2 设置跟踪类型 (SetTraceType)	57
3.6.4.3 设置跟踪点 (SetTracePnt)	57
3.7 封装接口函数	58
3.7.1 获取 FFT 频率 (GetFftFreq, MHz)	58
3.7.2 获取检测时间 (GetTstTime, ns)	58
3.7.3 获取检测距离 (GetTstRange, mm)	58
3.7.4 获取最大重复频率 (GetMaxRptFreq)	59
3.7.5 获取单块板卡无累积的采集双字数 (GetSoloBrdNoAccDaqDws)	59
3.7.6 初始化 FFT (FftInit)	60
3.7.7 执行 FFT (FftExec)	60
3.7.8 初始化实时存储 (InitRtMem)	60

第 4 章 ZXUT-NET 接口函数使用范例 63

4.1 核心参数 (ST_CORE_PARAM) 的数据结构描述	63
4.1.1 逻辑通道 (LogChan)	63
4.1.2 采样深度和分频比 (SmplDepth & FreqRatio)	64
4.1.3 数据开关 (DataSwitch)	64
4.1.3.1 采样波形 (SmplWave)	64
4.1.3.2 FFT 波形 (FftWave)	64
4.1.3.3 中心频率 (CentFreq)	65
4.1.4 通道时间&同步延迟占额	65
4.1.4.1 铁轨检测的设置范例	65
4.2 中间件导出信息 (ST_EXP_INFO) 的数据结构描述	67
4.2.1 版本信息 (VerInfo)	68
4.2.2 位宽信息 (BwInfo)	68
4.2.3 板号 (BrdSn)	69
4.2.4 配置信息	69
4.2.4.1 已连接通道板卡总数 (TotBrd)	69
4.2.4.2 板内硬通道总数 (TotHard)	69
4.2.4.3 每个硬通道包含的软通道总数 (TotSoft)	69
4.2.4.4 板内同步关系 (InnerSync)	70
4.2.4.5 通道板模拟电路的最大增益 (MaxGain)	70
4.2.4.6 通道板网卡 MAC 地址 (MacAddr)	70
4.2.4.7 采集缓冲区地址 (DaqBuf)	70
4.2.4.8 采集事件句柄地址 (DaqEvt)	70
4.2.5 系统时序解析结果	70
4.3 探伤参数的设置步骤	71
4.3.1 *** 打开探伤设备 (OpenDevice)	71
4.3.2 设置数字输入 (DI) 参数	77
4.3.3 设置数字输出 (DO) 参数	78
4.3.4 设置编码器 (ENC) 参数	79
4.3.5 *** 设置探伤参数 (SetFlawParam)	80
4.3.6 *** 设置重复频率 (SetRptFreq)	80
4.3.7 *** 创建逻辑通道表	80
4.3.8 *** 创建数据处理线程	80
4.4 探伤数据结构	81
4.4.1 距离补偿 (TCG) 参数	81
4.4.2 闸门判伤特征值 (结果)	81
4.4.3 累积缓冲区	82
4.4.4 采集缓冲区	82
4.5 数据处理线程	83

第 5 章 演示软件的使用说明 85

5.1 回波显示区	85
-----------------	----

5.2 右侧主面板	86
5.2.1 开关采样	86
5.2.2 客户机相关	86
5.2.3 A 闸门特征值	86
5.2.4 自动增益	86
5.2.5 波形导出	86
5.2.6 快捷键	87
5.2.7 关于本软件	87
5.3 调节参数选项卡	88
5.3.1 探伤参数选项卡	88
5.3.1.1 解析系统时序结果	88
5.3.1.2 增益粗调	89
5.3.1.3 自动增益	89
5.3.1.4 参数同步	89
5.3.1.5 探伤参数的调节	89
5.3.1.6 声程差	90
5.3.1.7 通道使能	90
5.3.2 探伤闸门选项卡	90
5.3.3 数字输入选项卡	91
5.3.4 数字输出选项卡	91
5.3.5 编码器选项卡	91
5.3.6 距离补偿 (TCG) 选项卡	92
5.3.7 系统设置选项卡	92
5.3.7.1 IP 地址设置	93
5.3.7.2 板卡当前温度	93
5.3.7.3 当前通道板的序号	93
5.3.7.4 系统板卡信息	93
5.3.8 FFT 处理选项卡	94
5.4 实时存储 (RtMem)	95
5.4.1 关闭采样, 设置实时存储参数	95
5.4.2 开启采样, 等待实时存储结束	96
5.4.3 实时存储数据的确认	96

第 1 章 硬件描述及技术指标

1.1 产品概述

ZXUT 系列多通道超声波探伤板卡采用工业 PCI 总线或千兆以太网接口, 包含多个超声波发射/接收模拟通道, 和高速 A/D 转换及数字控制、信号处理器集成在一起组成数模混合电路模块。通过先进的电路技术和多层 PCB 技术的融合, 以及全 SMT 安装工艺, 实现了高集成度的设计, 达到了更高的信噪比等性能指标和可靠性。

ZXUT 系列多通道超声波探伤板卡通过总线结构和计算机系统无缝集成, 用户可根据需要多卡级连, 构成更多通道超声波探伤系统。在多通道自动探伤、超声成像方面具有更多的速度优势和灵活性。适用于相关院校、研究所及企业在此基础上构造定制的超声波自动检测系统, 或超声成像系统, 也可由我司直接面向最终用户提供完整的检测系统。

ZXUT 系列多通道超声波探伤板卡提供设备驱动程序、SDK 动态链接库 (DLL) 以及源代码级的演示软件。用户可参照此演示软件构建定制的超声波检测软件。由于 SDK 动态链接库是标准的 C 语言 DLL, 用户选择多种编程语言, 如: C/C++、C#、BASIC、DELPHI、JAVA、LabView 以及 LabWindows 等等。为用户提供了极大的灵活性。

1.2 主要技术指标

- **通道数:** 4 个串行通道, 多块板卡可级联构建多通道超声波探伤系统;
- **工作模式:** 单晶模式、双晶模式;
- **检波模式:** 全检波、正检波、负检波和射频 (RF);
- **脉冲发射电压:** 20 ~ 300V 可调, 1V 步进;
- **重复频率:** 10 ~ 50KHz, 在此重复频率范围下, 均可处理显示动态缺陷回波;
- **A/D 频率及精度:** 100MHz, 8 位 (PCI) / 12 位 (NET);
- **触发模式:** 定时模式、编码器模式和外触发模式;
- **回波模式:** 自由模式、闸门 A、闸门 B 和闸门 C;
- **检测范围:** 0 ~ 10m (取决于重复频率);
- **增益:** 总共 110dB, 0.1dB 步进;
- **工作频率:** 0.5MHz ~ 25MHz
- **等效电噪声:** $80 \times 10^{-9} V / \sqrt{Hz}$;
- **灵敏度余量:** $\geq 60dB$ (200mm, $\Phi 2$ 平底孔);
- **垂直线性误差:** $\leq 3\%$;
- **水平线性误差:** $\leq 1\%$;
- **检测闸门:** 4 个硬件检测闸门, 包括: 伤波、底波、界面波以及用户闸门;
- **动态范围:** $\geq 30dB$;
- **板卡同步信号:** 每板卡具有同步输入 (SYN_IN) 和输出 (SYN_OUT) 接口;
- **旋转编码器:** 支持 4 个独立的 32 位编码器接口;
- **数字输入端口:** 8 个独立 DI, 包含硬件滤波整形硬件电路;
- **数字输出端口:** 8 个独立 DO, 可直接输出或者脉冲输出 (脉冲延迟和脉冲宽度可调);
- **发射脉冲宽度:** 50 ~ 1000ns 连续可调, 步进 10ns;

- **分辨力:** $\geq 40\text{dB}$ (5P14);
- **软件接口:** 提供二次开发软件接口 (包括驱动程序、动态库及演示软件);
- **距离补偿:** 具有大工件距离补偿 (TCG) 功能;
- **探伤参数:** 每个通道具有独立的探伤参数, 可单独调节与保存;
- **探头接口:** CC4 或 LEMO00 插座;
- **适用范围:** 本板卡可组成多通道超声波探伤系统, 广泛用于板材、棒材、管材 (无缝管、焊管) 以及坯材等金属材料的无损检测或超声波成像, 同时也可用于测厚。

1.3 PCI 板卡硬件构成

ZXUT-PCI 多通道超声波探伤卡硬件由以下几部分构成:

- 超声波探头插座;
- 超声波发射&接收模块;
- 模数转换;
- 高压模块;
- PCI 桥接控制器;
- 现场可编程门阵列 (FPGA);
- 温度监控;
- DI、DO 和 ENC;
- 同步输入&输出。

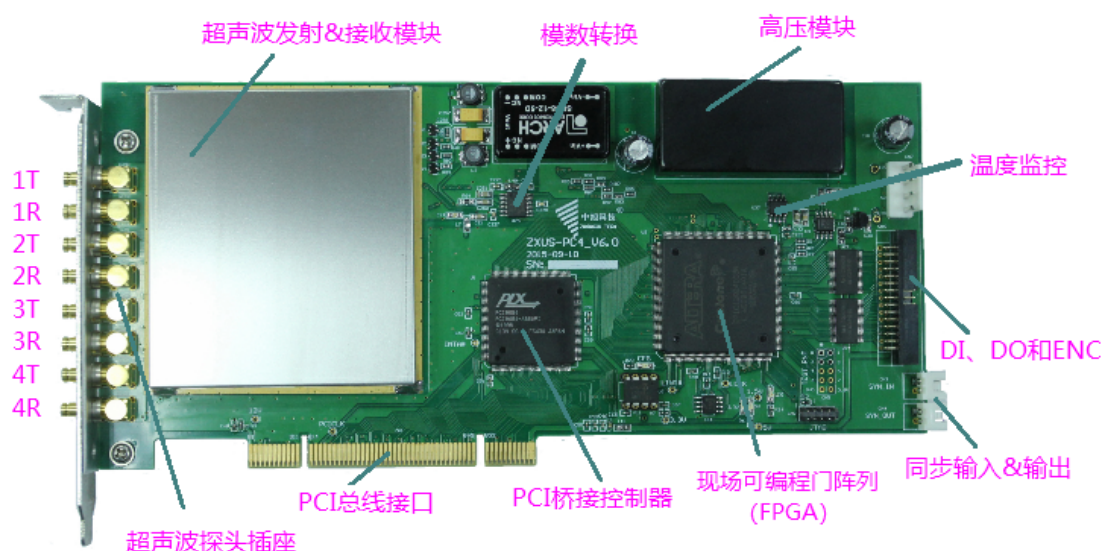


图 1-1: 超声波探伤卡实物

下面分别对上述几个部分进行简述。

1.3.1 超声波探头插座

采用 CC4 接插件, 我司随板提供 CC4 到 Q9 (BNC) 转换电缆。接驳的超声波探头从上到下依次为: 1T、1R、2T、2R、3T、3R、4T、4R。其中 T 表示发射, R 表示接收。

若使用单晶模式（即自发自收），请将探头接至 T 端；若使用双晶模式（即单发单收，又称一发一收），请将发射探头接至 T 端、接收探头接至 R 端。

1.3.2 超声波发射&接收模块

该模块实现所有的超声波板卡的模拟功能，包括：超声波发射、110dB 信号放大、带通滤波器等。该部分电路体现了超声波设备的主要技术指标。

1.3.3 模数转换

该部分电路将前级生成的模拟信号转换为数字信号，采样速率为 100MHz，送至后续的现场可编程门阵列（FPGA）中进行数字信号处理。

1.3.4 高压模块

该模块产生用于脉冲发射的可调高压，调节范围为 20V 到 300V，调节步进为 1V。该高压模块用于超声波发射电路的方波脉冲激励。

1.3.5 PCI 桥接控制器

该控制器芯片实现 CPU 总线与局部总线进行桥接，以便后级的现场可编程门阵列（FPGA）能够访问 CPU 总线，与 CPU 总线进行数据交换。

1.3.6 现场可编程门阵列（FPGA）

这是数字部分电路的核心，实现了数字检波、系统时序生成、极值保持、ADC 接口、闸门判伤等实时数字信号处理。

1.3.7 温度监控

数字信号处理器读取温度实时监控，以便配合模拟电路进行温度补偿。同时也避免温度过高而对电路板造成永久物理损坏。当前电路板的温度值实时显示在演示软件中。

1.3.8 DI、DO 和 ENC

该部分提供机电接口：包括 8 个数字输入（DI）、8 个数字输出（DO）和 4 个 32 位旋转编码器（ENC）接口。位于板卡右侧。

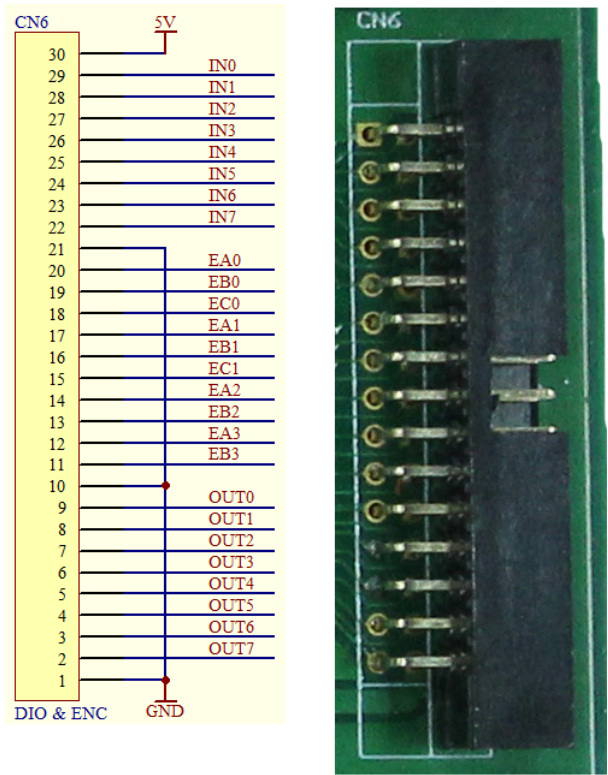


图 1-2：DIO 和 ENC 实物及图示

上述接口的电气定义（所有 I/O 均遵循 5V TTL 电平）如下：

管脚编号	信号名称	信号定义
22 ~ 29	IN[7:0]	数字输入（DI）
12, 14, 17, 20	EA[3:0]	编码器 A 相输入
11, 13, 16, 19	EB[3:0]	编码器 B 相输入
15, 18	EC[1:0]	编码器 Z 相输入
2 ~ 9	OUT[7:0]	数字输出（DO）

表 1-1：DIO 和 ENC 接口定义

1.3.9 同步输入与输出

同步输入（**SYN_IN**）和同步输出（**SYN_OUT**）实现多块板卡级联组件多通道超声波探伤系统。位于板卡右下角。

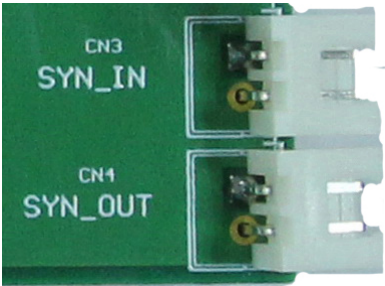


图 1-3：同步 IO 接口

第一次运行演示软件后，板卡右上角的主板指示灯若亮，表示该板为主板，其余板卡为从板。通过同步电缆连接主板和多块从板，主板的同步输出连接到从板的同步输入。

1.4 PCI 板卡硬件的安装

拆开探伤板卡的防静电袋包装，将板卡缓慢、均匀用力插入计算机主板空闲 PCI 插槽。用固定螺丝将板卡固定于计算机机箱。

特别注意：由于探伤板卡内含较多对静电非常敏感的数模混合集成电路，用于在拔插探伤板卡时，请勿触摸电路部分！避免人体静电对电路板上的集成电路造成永久物理损坏。

第 2 章 软件开发概述

ZXUT-NET 多通道超声波检测系统是基于千兆以太网、由多达 8 个设备机箱堆叠而成的超声波检测系统。每个设备机箱由多达 10 块超声波板卡组成，每块板卡包含 4 个串行硬件通道，每个硬件通道包含 4 个软件通道。板卡之间可以串行、也可以并行。

2.1 内容描述

本软件开发指南对 ZXUT 系列多通道超声波检测系统的**软件编程环境、运行文件组成及描述、软件分层模型、中间件完全 API 函数调用说明**以及**演示软件的实现**等内容进行了详尽描述。

特别注意：目前的驱动程序、中间件 DLL 及应用程序可以在 32 位或 64 位操作系统上运行，包括 Windows 2000、Windows XP 以及 Windows 7/8/10 32 位或 64 位操作系统。

2.2 软件分层模型

软件分层模型的框图如图 2-1 所示：

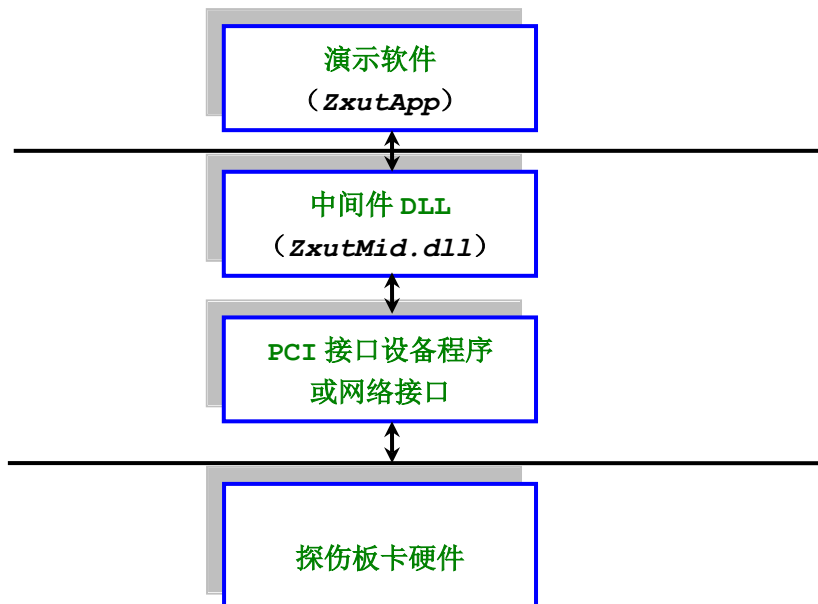


图 2-1：软件分层模型

2.3 演示软件

演示软件 ZxutApp.exe 运行于 Windows 32 位或 64 位平台，主要向软件开发人员演示如何设置通道，如何获取通道采样数据（包括采样波形以及用户闸门内峰值和前沿信息）

以及如何设置各种通道参数（比如：采样深度、发射脉宽、dB 码、检测范围）。

演示软件任何有关 ZXUT 设备相关的访问控制，均是通过中间件来实现的。如此，软件开发人员不需了解任何设备实现细节，即可实现对设备的全面访问控制。

演示软件基于微软的 VisualStudio 2015 开发平台，由于演示软件和中间件 DLL 均采用**静态链接**的形式，无需安装任何的 vs2015 的运行库或安装包。

2.3.1 探伤参数文件

探伤参数文件为 `FlawParam.zxut`，若将该文件删除，演示软件会将探伤参数初始化，恢复到缺省值。

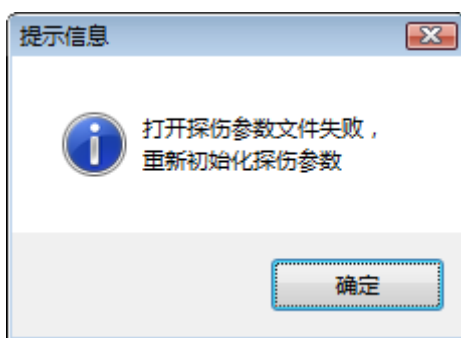


图 2-2：探伤参数初始化

2.4 中间件

中间件 `ZxutMid.dll` 是一个标准的动态链接库（DLL），供用户通过编程语言调用。中间件在整个分层软件模型中起着承上启下的作用。往上，接受从用户程序发来的指令，并反馈采样数据及参数给用户程序；往下，将用户指令和探伤参数发送给设备软件系统。中间件开放了所有 ZXUT 探伤设备所支持的软件 API 函数接口。

`ZxutMid.lib` 和 `ZxutMid.dll` 文件位于：

■ 调试版本：

- 。 32 位 `ZxutApp/Product/32/Debug`
- 。 64 位 `ZxutApp/Product/64/Debug`

■ 发布版本：

- 。 32 位 `ZxutApp/Product/32/Release`
- 。 64 位 `ZxutApp/Product/64/Release`

由于 `ZxutMid.dll` 采用标准的 DLL 接口，因此编程语言并不局限于 C++，也可使用 C#、LabView、Qt、JAVA、BASIC 等等。

2.5 ZXUT-NET 软件编程

ZXUT-NET 的软件系统全部基于**客户机/服务器**（**C/S**: Client/Server）模式实现。设备作为客户端，而主机作为服务器端。两者之间通过 1GHz 以太网连接起来。

2.5.1 客户机软件

客户机系统环境采用 Linux 操作系统。客户机源代码全部使用标准 C 语言编写，通过 gcc 编译生成执行程序。客户机的缺省 IP 地址为：**10.0.0.30**，可通过 API 函数进行修改。可能存在多个客户机，上述地址为它们的**首地址**。

2.5.2 服务器软件

服务器软件源代码全部使用标准 C++ 语言编写，编译环境使用微软的 Visual Studio .Net 2015 编程实现。服务器的缺省 IP 地址为：**10.0.0.100**，可通过 API 函数进行修改。

事实上，服务器可以使用任何支持网络通讯的操作系统平台，并不局限于演示软件所使用的 Windows 平台。还包括 Unix、Linux、MacOs 等现代操作系统。这就是采用 CS 模式的好处，通过网络连接，将网络两端的操作系统完全隔离开来。用户可以根据自己的喜好以及对 OS 的熟悉程度来选择编程系统平台。

2.6 ZXUT-PCI 驱动程序及安装

PCI 驱动程序文件包含系统文件 ZxutDrv.sys 以及信息文件 ZxutDrv.inf。安装步骤如下：

- （1）先安装探伤板卡，再打开计算机电源开机。
- （2）操作系统完成启动，出现如下对话框：

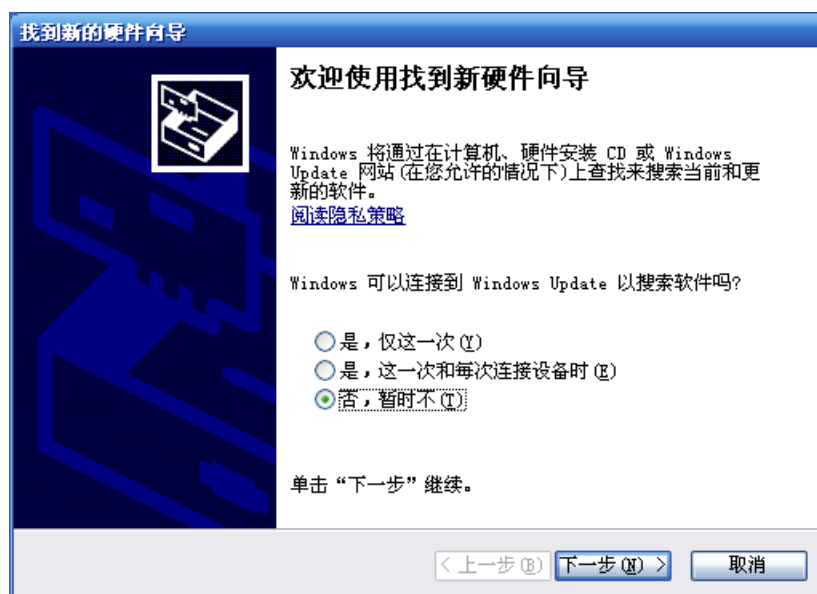


图 2-3: 找到新的硬件向导 1

(3) 在上述对话框中, 选择“否, 暂时不 (N)”, 然后再单击“下一步”按钮。出现如下对话框:

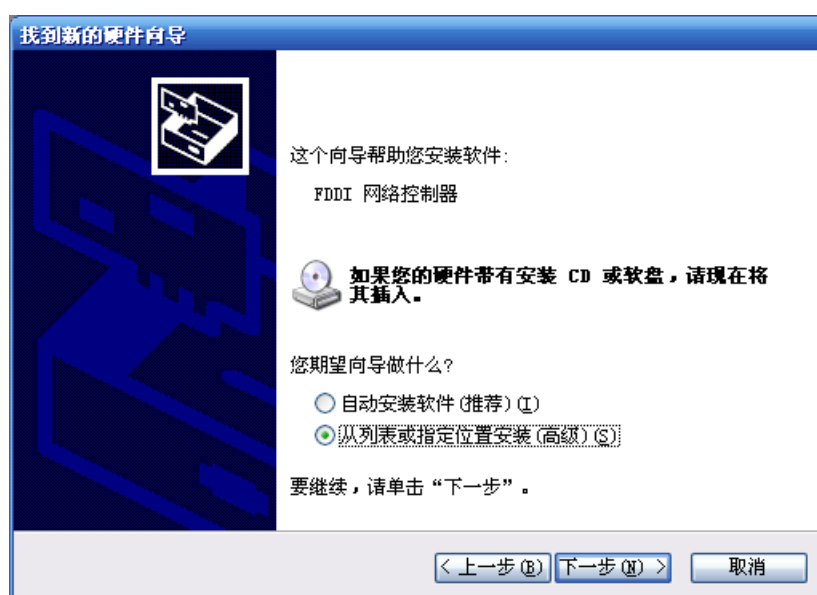


图 2-4: 找到新的硬件向导 2

(4) 在上述对话框中, 选择“从列表或指定位置安装 (高级) (S)”, 然后再单击“下一步”按钮。出现如下对话框:

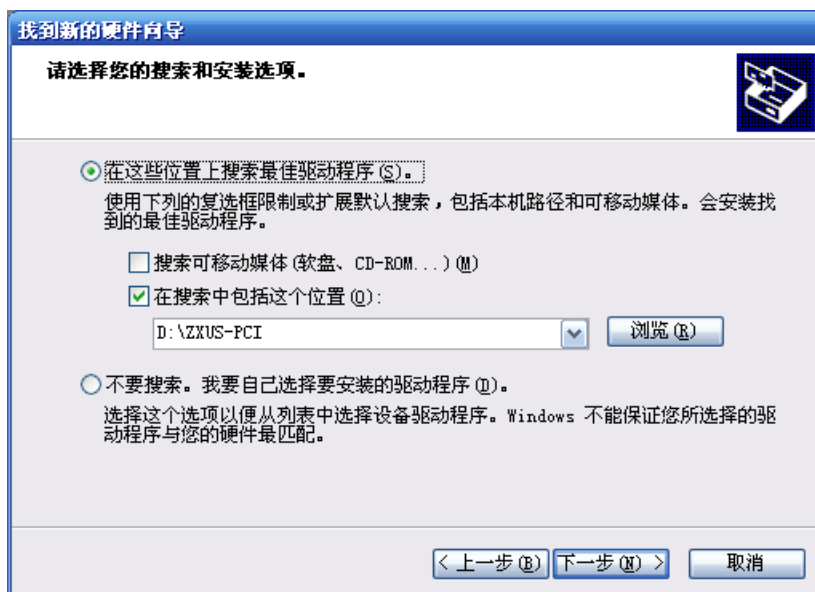


图 2-5：找到新的硬件向导 3

(5) 在上述对话框中，单选“在搜索中包含这个位置(O)”，再选择 ZXUS-PCI 软件所在的文件夹 **D:\ZXUT-PCI**（以实际存储软件的文件夹位置为准），然后单击“下一步”按钮。出现如下对话框：



图 2-6：找到新的硬件向导 4

(6) 出现上述对话框，表示“ZXUT-PCI 多通道超声波探伤板卡”成功安装完成。单击“完成”按钮。

(7) 右击“我的电脑”，选择“属性”。在出现的“系统属性”对话框中，单击“硬件”选项卡。再单击“设备管理器”。出现如下对话框：

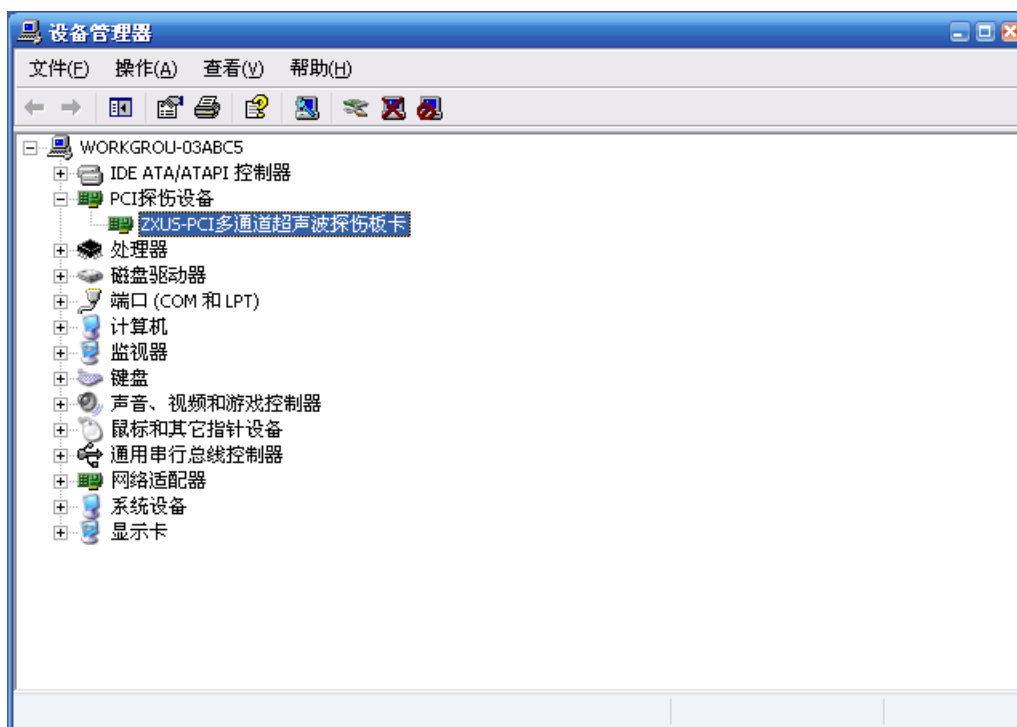


图 2-7：设备管理器

(8) 在上述对话框中，右击选中“ZXUT-PCI 多通道超声波探伤板卡”属性，出现如下对话框：

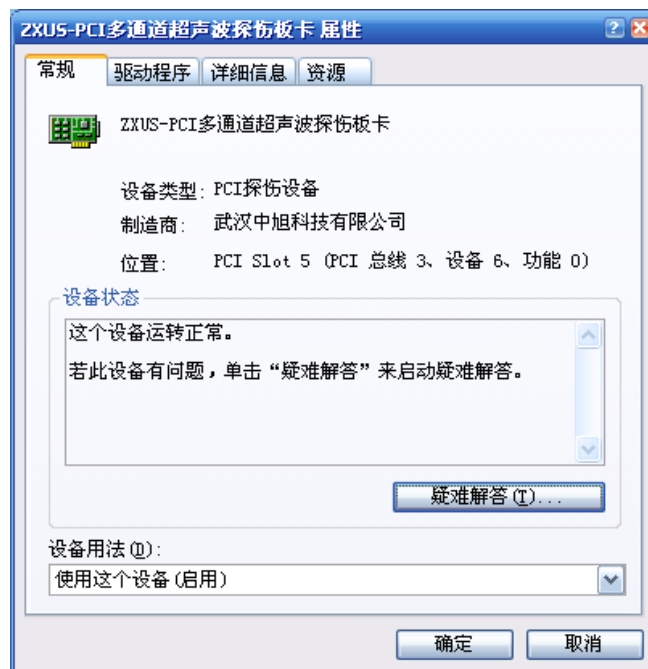


图 2-8：成功安装的板卡属性

(9) 至此，“ZXUS-PCI 多通道超声波探伤板卡”驱动程序安装完成。

2.7 Windows 64 位操作系统的特别考虑

Windows 64 位操作系统，比如 Windows 7 64 位和 Windows 10 64，均要求驱动程序提供数字签名。对于小众工业设备的驱动程序，微软提供测试签名。但前提是，要将操作系统设置为测试模式。

2.7.1 运行命令行

以管理员身份，运行命令行。

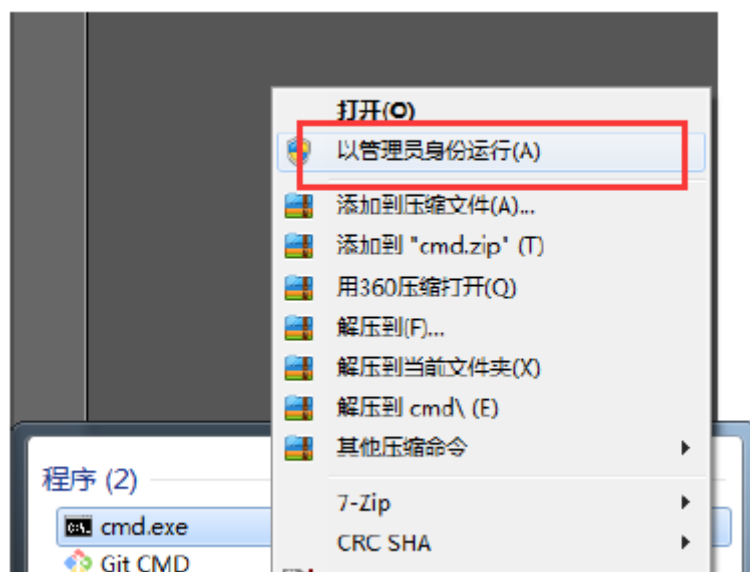


图 2-9：管理员身份运行 cmd.exe

2.7.2 设置操作系统为测试模式

在打开的命令行中，输入：

```
Bcdedit.exe -set TESTSIGNING ON
```

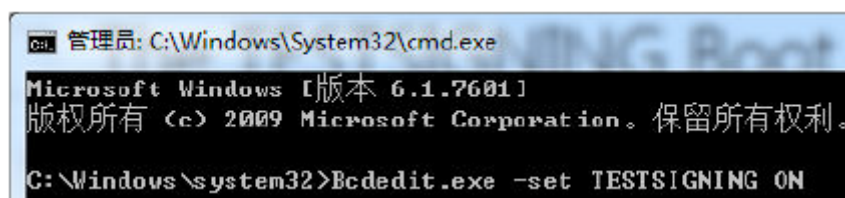


图 2-10：设置测试模式

2.7.3 重启操作系统

重启系统后，将进入测试模式。可以安装测试签名的驱动程序。

2.8 常用缩略语

为了简化编码，在本软件设计中采用了大量的缩略语。为了便于软件开发人员了解并熟知这些缩略语，将常用的缩略语列举如下：

缩略语	全称	描述
zxut	zexu ultrasonic testing	泽旭超声波检测
cmn	common	共用的、共享的
net	network	网络
ver	version	版本
dev	device	设备
drv	driver	驱动程序
chan	channel	通道
det	detective	侦测
trans	transfer	传输
buf	buffer	缓冲区、缓冲器
ptr	pointer	指针
enb	enable	使能、有效
dis	disable	禁止、无效
crrnt	current	当前的
int	interrupt	中断
dcft	defect	缺陷、伤波
bttm	bottom	底波
intf	inetrface	界面波
usr	user	用户
hard	hardware	硬件
soft	software	软件
imp	impedance	阻抗
rf	radio frequency	射频
rej	rejection	抑制
pul	pulse	脉冲
freq	frequency	频率
rt	receive/transmit	收发
dem	demodulate	检波
orig	origin	零位
dly	delay	延迟
rpt	repeat	重复
smpl	sample	采样
acc	accumulate	累积
enc	encoder	编码器

表 2-1：常用缩略语

缩略语	全称	描述
reg	register	寄存器
dig	digital	数字的
ang	analog	模拟的
sys	system	系统
info	information	信息
bw	bit width	位宽
sig	signature	标识
app	application	应用、应用程序
fltr	filter	滤波器
pol	polarity	极性
clr	clear	清除
res	result	结果
avg	average	平均
cmd	command	命令
di	digital input	数字输入
do	digital output	数字输出
max	maximum	最大
min	minimum	最小
cfg	configuration	配置

表 2-1: 常用缩略语 (续)

第 3 章 中间件（DLL）接口命令详述

特别注意: 为了提高参数设置的效率,并没有在中间件中过多地对输入参数的有效性(即满足最小值和最大值的条件)进行检查。使用该中间件的程序员必须自行确保调用函数的输入参数是有效的。对于无效的输入参数调用,其函数执行的结果是不可预知的。可能什么也没有影响,也可能对系统的稳定性产生消极影响,在严重的极限情况下,甚至可能导致系统复位或者死机。

3.1 包含头文件

用户应用程序需要包含中间件头文件 **ZxutCmn.h** 以及 API 接口函数的头文件 **ZxutExp.h**。

该头文件中定义了常用数据类型(比如 **U8**、**U32** 等),一些相关常量(比如通道数等等),最重要的是内核层反馈给应用层的数据结构,其中又包含每个通道的采样波形数据。

3.2 ZxutCmn.h 包含内容简述

3.2.1 常用数据类型定义

```
////////////////////////////////////  
//  
// 常用数据类型定义  
//  
////////////////////////////////////  
  
typedef unsigned char    U8;           // 无符号位整型变量  
typedef signed   char    S8;           // 有符号位整型变量  
  
typedef unsigned short   U16;          // 无符号位整型变量  
typedef signed   short   S16;          // 有符号位整型变量  
  
typedef unsigned int     U32;          // 无符号位整型变量  
typedef signed   int     S32;          // 有符号位整型变量  
  
typedef unsigned long long U64;        // 无符号64位整型变量  
typedef signed   long long S64;        // 有符号64位整型变量  
  
// 在驱动程序中,不提供浮点数的支持  
typedef float          F32;            // 单精度浮点数(32位长度)  
typedef double         F64;            // 双精度浮点数(64位长度)
```

3.2.2 版本号

```
#define ZXUT_NET_PCB_VER 3 // [4] NET PCB版本号
#define ZXUT_PCI_PCB_VER 7 // [4] PCI PCB版本号
#define ZXUT_FPGA_VER ((2 << 8) + (2)) // [12 = 4 + 8] FPGA版本号
#define ZXUT_DEV_VER ((1 << 8) + (55)) // 设备版本
```

共 16 位，高 8 位表示主版本号，低 8 位表示次版本号。比如 v1.55，主版本号为 1，次版本号为 55。每个交付给用户的中间件均有一个版本号，用户用于区分 ZxutMid.dll 的软件版本，确认软件功能。

3.3 ZxutExp.h 包含内容简述

此文件包含所有支持的 API 的函数原型。本章后续提供的文档，仅局限于各种 API 函数的参数说明，关于该函数的具体使用，会在第 3 章以实例的形式提供。

3.3.1 设备制造商专用 API 函数

该文件中标注为：

```
// *** 设备制造商专用，最终用户请勿调用***
```

的 API 函数，仅为设备制造商测试使用。未来可能存在极大的变动，且不会提供任何文档，用户请勿调用这些 API 函数。

3.3.2 API 函数返回码

```
////////////////////////////////////
// API函数返回码(Zxut Return Code, ZRC)

typedef enum _ET_ZRC
{
    ZRC_API_RET_SUCCESS = 0, // API函数返回成功
    ...
} ET_ZRC;
```

以枚举的形式定义了 API 函数返回码，具体定义参见 ZxutCmn.h 文件。后续会对这些返回值进行详细描述。

3.4 设备接口函数

本节中列举的设备接口函数与探伤参数无关，仅牵涉到整个 ZXUT 设备中非探伤参数的

接口部分。

后续 API 函数中出现的 nBrdNum 参数均遵循：

U8 nBrdNum [IN] 板卡编号，从 0 到 MAX_BRD_NUM - 1

在 API 函数的参数描述中不再复述。

```
#define MAX_BRD_NUM                   10                   // 支持的最多通道板卡数
```

若系统中存在 4 块板卡，板号的取值范围为：0 ~ 3。后续函数中，若板号超出该范围，则会得到返回值：ZRC_INVALID_BRD_NUM。

3.4.1 打开探伤设备 (OpenDevice)

函数原型：

```
ET_ZRC ZXUT_OpenDevice(U8 nZxutType, U8 nTotBrd,
                        SOCKET sockZxut[MAX_BRD_NUM], ST_EXP_INFO** ppstExtInfo);
```

函数参数：

U8 nZxutType [IN] 探伤设备类型
 U8 nTotBrd [IN] 板卡总数，仅针对 ZXUT-NET
 SOCKET sockZxut [IN] 网络套接字，仅针对 ZXUT-NET
 ST_EXP_INFO** ppstExtInfo [OUT] 导出信息双指针

功能描述：

该函数打开探伤设备，并给出导出信息双指针。对于 ZXUT-NET，会将已经连接的网络套接字传递给 DLL。

可能的探伤设备类型为：

```
#define ZXUT_TYPE_NET           0    // NET设备类型
#define ZXUT_TYPE_PCI          1    // PCI设备类型
```

可能的错误返回值为：

- ZRC_INVALID_ZXUT_TYPE，无效的探伤设备类型
- ZRC_NO_BRD_INSERT，没有找到 PCI 探伤板卡；
- ZRC_SYS_INFO_MISMATCH，反馈的系统信息不全相同；
- ZRC_INVALID_PCB_VER，无效的 PCB 版本；
- ZRC_INVALID_FPGA_VER，无效的 FPGA 版本；
- ZRC_INVALID_TOT_HARD，无效的硬通道总数；
- ZRC_INVALID_TOT_SOFT，无效的软通道总数。

关于 ST_EXP_INO 的定义，请参见后面数据结构定义。

3.4.2 关闭探伤设备 (CloseDevice)

函数原型：

```
ET_ZRC ZXUT_CloseDevice(U8 nAppExitCmd);
```

函数参数：

U8 nAppExitCmd [IN] 用户软件退出时，发送给网络探伤设备的退出命令。

对于 PCI 探伤设备，任意指定

```
////////////////////////////////////
// 应用程序退出时，给客户机发送的命令
```

```
#define APP_EXIT_CMD_RETRY      0      // 客户机重试
#define APP_EXIT_CMD_EXIT      1      // 客户机退出
#define APP_EXIT_CMD_REBOOT    2      // 客户机重启
#define APP_EXIT_CMD_HALT      3      // 客户机关机
```

功能描述：

关闭探伤设备，并对网络探伤设备运行状态给出指示：(1) 重试，等待服务器连接；(2) 退出，不再连接服务器；(3) 重启，复位客户机系统；(4) 关机，客户机关闭，用户需要关闭设备电源。

可能的错误返回值为：

■ ZRC_INVALID_APP_EXIT_CMD，无效的设备退出命令

3.4.3 设置网络配置 (SetNetCfg)

函数原型：

```
ET_ZRC ZXUT_SetNetCfg(U8 nBrdNum, U8 anIpAddr[5], U32 nNetPort);
```

参数描述：

U8 anIpAddr[5] [IN] 客户机和服务器的 IP 地址

- 。 [0]：IP 地址的第 1 个字段，缺省值为 10；
- 。 [1]：IP 地址的第 2 个字段，缺省值为 0；
- 。 [2]：IP 地址的第 3 个字段，缺省值为 0；
- 。 [3]：客户机 IP 首地址的第 4 个字段，缺省值为 30；
- 。 [4]：服务器 IP 地址的第 4 个字段，缺省值为 100。

U32 nNetPort [IN] 网络通讯端口，缺省值为 8020。

功能描述：

设置客户机的 IP 首地址（缺省值为 10.0.0.30）、服务器的 IP 地址（缺省值为

10.0.0.100) 以及网络端口 (缺省值为 8020)。

若使用上述网络配置缺省值, 无需调用该函数。

仅针对 ZXUT-NET。

3.4.4 获取探伤通道板序列号 (GetBrdSn)

函数原型:

```
ET_ZRC ZXUT_GetBrdSn(U8 nBrdNum, U8 nBrdSnType);
```

函数参数:

U8 nBrdSnType [IN] 板卡的序列号类型

```
////////////////////////////////////  
// 板序列号类型  
  
#define BRD_SN_TYPE_CORE       0        // 核心板  
#define BRD_SN_TYPE_CHAN       1        // 通道板  
#define BRD_SN_TYPE_HV         2        // 高压模块  
#define BRD_SN_TYPE_PWR        3        // 电源模块  
  
#define BRD_SN_TYPE_NUM        4        // 板号个数  
  
#define BRD_SYN_TYPE_LEN       5        // 每种板序列号占个5字节, 10位数字
```

功能描述:

该函数返回探伤板卡的板序列号, 用户可能需要该序号进行软件加密。

若不使用板卡序列号, 无需调用该函数。

可能的错误返回值为:

■ **ZRC_INVALID_BRD_SN_TYPE**, 无效的板卡序列号类型

3.4.5 设置核心参数指针 (SetCoreParamPtr)

函数原型:

```
void ZXUT_SetCoreParamPtr(ST_CORE_PARAM* pstCoreParam);
```

函数参数:

ST_CORE_PARAM* pstCoreParam [IN] 核心参数指针

功能描述:

关于核心参数数据结构，参见后续章节。

3.4.6 设置最大采集率 (SetMaxDaqRate)

函数原型:

```
void ZXUT_SetMaxDaqRate(U32 nMaxDaqRate);
```

函数参数:

U32 nMaxDaqRate [IN] 最大采集率

功能描述:

采集率即为探伤设备往应用程序发送采集数据的频率。

最大数据采集率定义如下:

```
#define MAX_DAQ_RATE (5 * 1000) // 最大采集率 5KHz
```

3.5 系统参数接口函数

本节中列举的接口函数与设备系统参数有关，牵涉到：通道使能、采样使能以及 I/O 端口和编码器接口函数等。

3.5.1 通用系统参数接口函数

3.5.1.1 设置采样使能 (SetSmplEnb)

函数原型:

```
void ZXUT_SetSmplEnb(BOOL bSmplEnb);
```

函数参数:

BOOL bSmplEnb [IN] 采样使能，0/1

功能描述:

nSmplEnb = 1，使能采样；nSmplEnb = 0，禁止采样。采样使能后，开始发射/接收超声波信号，同时探伤设备将捕获采集数据。

3.5.1.2 设置发射电压 (SetTransVol)

函数原型:

```
ET_ZRC ZXUT_SetTransVol(U8 nBrdNum, U16 nTransVol);
```

函数参数:

U16 nTransVol [IN] 发射电压

功能描述:

该函数可设置发射电压。其中 nTransVol 的取值范围为:


```

////////////////////////////////////
// 发射电压

```

```

const U16 TRANS_VOL_MIN      = 20;      // 20V
const U16 TRANS_VOL_MAX      = 300;     // 300V

```

可能的错误返回值为：

■ **ZRC_INVALID_TRANS_VOL**，无效的发射电压

3.5.1.3 设置触发模式(SetTrigMode)

函数原型：

```

ET_ZRC ZXUT_SetTrigMode(U8 nBrdNum, U8 nTrigMode);

```

函数参数：

```

U8 nTrigMode      ..... [IN] 触发模式

```

功能描述：

目前系统支持三种触发模式，即：

。 **TRIG_MODE_TIME**：定时模式（缺省模式）；

nTrigMode = 0，FPGA 硬件根据重复频率的设置采集数据。比如系统设定重复频率设定为 500Hz，则每 2ms 客户机将超声采集数据通过网络传输给计算机系统。

。 **TRIG_MODE_ENC**：编码模式。

nTrigMode = 1，根据服务器软件设定的扫描间隔（或称扫描距离），FPGA 硬件会将最靠近指定编码器位置的超声数据传输给计算机系统。

。 **TRIG_MODE_EXT**：外部模式。

nTrigMode = 2，触发信号由设备外部给出。

```

////////////////////////////////////
// 触发模式

```

```

#define TRIG_MODE_TIME    0      // 定时触发
#define TRIG_MODE_ENC     1      // 编码触发
#define TRIG_MODE_EXT     2      // 外部触发

```

可能的错误返回值为：

■ **ZRC_INVALID_TRIG_MODE**，无效的触发模式

3.5.1.4 设置触发极性(SetTrigPol)

函数原型:

```
ET_ZRC ZXUT_SetTrigPol(U8 nBrdNum, U8 nTrigPol);
```

函数参数:

```
U8 nTrigPol          ..... [IN] 触发极性
```

功能描述:

支持两种触发极性，即：

- **POL_DEF_NEG**：负极性，负脉冲或低电平有效；
- **POL_DEF_POS**：正极性，正脉冲或高电平有效（缺省值）；

```
////////////////////////////////////
// 极性定义
```

```
#define POL_DEF_NEG      0      // 负极性
#define POL_DEF_POS      1      // 正极性
```

可能的错误返回值为：

■ **ZRC_INVALID_POL_DEF**，无效的极性定义

3.5.1.5 设置扫描编码器 (SetScanEnc)**函数原型:**

```
ET_ZRC ZXUT_SetScanEnc(U8 nBrdNum, U8 nScanEnc);
```

函数参数:

```
U8 nScanEnc          ..... [IN] 编码器索引，0 ~ 3
```

功能描述:

设置哪个编码器作为扫描编码器，从 4 个编码器中选择。

可能的错误返回值为：

■ **ZRC_INVALID_ENC_CHAN**，无效的编码器通道

3.5.1.6 设置扫描间隔 (SetEncSpan)**函数原型:**

```
ET_ZRC ZXUT_SetScanSpan(U8 nBrdNum, U32 nScanSpan);
```

函数参数:

```
U32 nScanSpan        ..... [IN] 扫描间隔（20 位）
```

功能描述:

假定系统要求每 5mm 采集一次超声数据，根据编码器计数和行走距离的换算关系，得

到 5mm 对应 50 个编码计数。则将 nScanSpan 设定为 49 即可，FPGA 内部计数范围为 0 ~ 49，总编码器脉冲个数为 50 个。

可能的错误返回值为：

- **ZRC_INVALID_SCAN_SPAN**，无效的扫描间隔

3.5.1.7 设置板间同步模式 (SetBrdSync)

函数原型：

```
ET_ZRC ZXUT_SetBrdSync(U8 nBrdNum, U8 nBrdSync);
```

函数参数：

U8 nBrdSync [IN] 板间同步模式

功能描述：

目前系统支持两种板间同步模式。

```
//////////////////////////////////////
// 板间同步
```

```
#define BRD_SYNC_SERI    0      // 通道板之间为串行同步
#define BRD_SYNC_PARA    1      // 通道板之间为并行同步
```

在大多数应用场合，均使用并行同步模式。因为串行同步模式会极大地降低探伤系统的重复频率。

可能的错误返回值为：

- **ZRC_INVALID_BRD_SYNC**，无效的板间同步
- **ZRC_ZXUT_NOT_SUPPORT**，当前设备不支持，只有 ZXUT-NET-S4 支持

3.5.1.8 设置重复频率 (SetRptFreq)

函数原型：

```
ET_ZRC ZXUT_SetRptFreq(U32 nRptFreq, U8* pnHardNum, U8* pnSoftNum);
```

函数参数：

U32 nRptFreq [IN] 需要设置的重复频率
U8* pnHardNum [OUT] 通道时间不够的硬通道编号
U8* pnSoftNum [OUT] 通道时间不够的软通道编号

功能描述：

目前系统支持的重复频率范围为：

```
//////////////////////////////////////
```

// 重复频率 (Hz)

```
#define RPT_FREQ_MIN      1          // 最小1Hz
#define RPT_FREQ_MAX     50000      // 最大50kHz
```

可能的错误返回值为:

- **ZRC_INVALID_RPT_FREQ**, 无效的重复频率;
- **ZRC_NO_ENOUGH_CHAN_TIME**, 通道时间不够。具体通道编号在 `pnHardNum` 和 `pnSoftNum` 中指明。

3.5.2 编码器 (ENC) 参数接口

设定编码器相关参数, 包括模式、滤波、使能、类型及极性等等。

关于编码器通道参数, 均遵循:

U8 `nEncChan` **[IN]** 编码器通道, 0 ~ 3

超出该取值范围, 将得到如下返回值:

- **ZRC_INVALID_ENC_CHAN**, 无效的编码器通道

在 API 函数的参数描述中不再复述。

3.5.2.1 设置编码器滤波 (SetEncFltr)

函数原型:

```
ET_ZRC ZXUT_SetEncFltr(U8 nBrdNum, U8 nEncChan, U8 nEncFltr);
```

函数参数:

U8 `nEncFltr` **[IN]** 编码器滤波

功能描述:

可能的编码器滤波值列举如下, 指定编码器 A/B 相信号的最大频率。

```
////////////////////////////////////
// 编码器滤波
```

```
#define ENC_FLTR_10KHZ      0
#define ENC_FLTR_20KHZ      1
#define ENC_FLTR_50KHZ      2
#define ENC_FLTR_100KHZ     3
#define ENC_FLTR_200KHZ     4
#define ENC_FLTR_500KHZ     5
```

```
#define ENC_FLTR_NUM          6
```

超出该取值范围，将得到如下返回值：

■ **ZRC_INVALID_ENC_FLTR**，无效的编码器滤波值

3.5.2.2 设置编码器 AB 相计数器 (SetEabCntr)

函数原型：

```
ET_ZRC ZXUT_SetEabCntr(U8 nBrdNum, U8 nEncChan, U32 nEabCntr);
```

函数参数：

U32 nEabCntr [IN] 编码器 AB 相计数值

功能描述：

将32位编码值设置到指定编码器通道的AB相计数器，其初始值设定为0。

3.5.2.3 设置编码器 Z 相计数器 (SetEzCntr)

函数原型：

```
ET_ZRC ZXUT_SetEzCntr(U8 nBrdNum, U8 nEncChan, U32 nEzCntr);
```

函数参数：

U32 nEzCntr [IN] 编码器 Z 相计数值

功能描述：

将32位编码值设置到指定编码器通道的Z相计数器，其初始值设定为0。

3.5.2.4 设置编码器使能 (SetEncEnb)

函数原型：

```
ET_ZRC ZXUT_SetEncEnb(U8 nBrdNum, U8 nEncChan, BOOL bEncEnb);
```

函数参数：

BOOL bEncEnb [IN] 编码器使能

功能描述：

写 1 到该接口，使能对应通道的编码器（缺省值）；写 0，则禁止该通道的编码器。

3.5.2.5 设置编码器类型 (SetEncType)

函数原型：

```
ET_ZRC ZXUT_SetEncType(U8 nBrdNum, U8 nEncChan, U8 nEncType);
```

函数参数：

U8 nEncType [IN] 编码器类型（2 位）

功能描述:

设定编码器的计数类型。

- **ENC_TYPE_SOLO**: 单脉冲计数模式, A 作为计数脉冲, 与 B 无关;
- **ENC_TYPE_TWIN**: 双脉冲计数模式, A 作为计数脉冲, B 作为方向脉冲;
- **ENC_TYPE_QUAD**: 四脉冲计数模式, A/B 的上升/下降沿均计数 (缺省值)。

```

////////////////////////////////////
// 编码器类型

#define ENC_TYPE_SOLO    0    // 单脉冲
#define ENC_TYPE_TWIN    1    // 双脉冲
#define ENC_TYPE_QUAD    2    // 四脉冲

```

超出该取值范围, 将得到如下返回值:

■ **ZRC_INVALID_ENC_TYPE**, 无效的编码器类型

3.5.2.6 设置编码器极性 (SetEncPol)**函数原型:**

```
ET_ZRC ZXUT_SetEncPol (U8 nBrdNum, U8 nEncChan, U8 nEncPol);
```

函数参数:

U8 nEncPol [IN] 编码器极性 (1 位)

功能描述:

设定编码器的计数极性。

- **POL_DEF_NEG**: 负极性, B 为计数脉冲, A 为方向脉冲;
- **POL_DEF_POS**: 正极性, A 为计数脉冲, B 为方向脉冲 (缺省值);

```

////////////////////////////////////
// 极性定义

#define POL_DEF_NEG      0      // 负极性
#define POL_DEF_POS      1      // 正极性

```

可能的错误返回值为:

■ **ZRC_INVALID_POL_DEF**, 无效的极性定义

3.5.2.7 设置编码器回零使能 (SetEncZe)**函数原型:**

```
ET_ZRC ZXUT_SetEncZe (U8 nBrdNum, U8 nEncChan, BOOL bEncZe);
```

函数参数:

BOOL bEncZe **[IN]** 编码器回零使能 (1 位)

功能描述:

若设置了编码器回零使能, 则检测编码器 Z 相脉冲时, 将编码器 AB 相计数器自动清零。

3.5.3 数字输入 (DI) 参数接口

设定数字输入相关参数, 包括使能、模式、极性、宽度及延迟等等。

关于数字输入通道参数, 均遵循:

U8 nDiChan **[IN]** DI 通道, 0 ~ 7

超出该取值范围, 将得到如下返回值:

■ **ZRC_INVALID_DI_CHAN**, 无效的 DI 通道

在 API 函数的参数描述中不再复述。

3.5.3.1 设置数字输入使能 (SetDiEnb)

函数原型:

```
ET_ZRC ZXUT_SetDiEnb(U8 nBrdNum, U8 nDiChan, BOOL bDiEnb);
```

函数参数:

BOOL bDiEnb **[IN]** DI 使能 (1 位)

功能描述:

写 1 到该接口, 使能对应通道的 DI (缺省值); 写 0, 则禁止该通道的 DI。只有在输入使能的情况下, 才允许对输入的数字信号进行判断处理。

3.5.3.2 设置数字输入模式 (SetDiMode)

函数原型:

```
ET_ZRC ZXUT_SetDiMode(U8 nBrdNum, U8 nDiChan, U8 nDiMode);
```

函数参数:

U8 nDiMode **[IN]** DI 模式 (1 位)

功能描述:

设定数字输入的模式。

- **DIO_MODE_PUL**: 脉冲模式, 检测输入脉冲, 可以指定多种参数;
- **DIO_MODE_LEV**: 电平模式, 缺省模式, 直接检测现在的电平高低。

```

////////////////////////////////////
// DIO模式

#define DIO_MODE_PUL      0      // 脉冲模式
#define DIO_MODE_LEV      1      // 电平模式

```

超出该取值范围，将得到如下返回值：

■ **ZRC_INVALID_DIO_MODE**，无效的 DI 模式

3.5.3.3 设置数字输入极性(SetDiPol)

函数原型：

```
ET_ZRC ZXUT_SetDiPol(U8 nBrdNum, U8 nDiChan, U8 nDiPol);
```

函数参数：

U8 nDiPol [IN] DI 极性（1 位）

功能描述：

设定数字输入的极性。

- 。 **POL_DEF_NEG**：负极性，负脉冲或低电平；
- 。 **POL_DEF_POS**：正极性，正脉冲或高电平（缺省值）；

```

////////////////////////////////////
// 极性定义

#define POL_DEF_NEG      0      // 负极性
#define POL_DEF_POS      1      // 正极性

```

可能的错误返回值为：

■ **ZRC_INVALID_POL_DEF**，无效的极性定义

3.5.3.4 设置数字输入清除(SetDiClr)

函数原型：

```
ET_ZRC ZXUT_SetDiClr(U8 nBrdNum, U8 nDiChan);
```

函数参数：

略去。

功能描述：

清除数字输入。写该接口即将输入检测结果清零，无论写入的是 1 还是 0。

特别注意：仅对脉冲模式有效。

3.5.3.5 设置数字输入宽度(SetDiWidth)

函数原型：

```
ET_ZRC ZXUT_SetDiWidth(U8 nBrdNum, U8 nDiChan, U16 nDiWidth);
```

函数参数：

```
U16 nDiWidth    ..... [IN] DI 宽度（10 位）
```

功能描述：

设置数字输入脉冲持续的宽度。外部信号必须满足（持续时间长于）这个检测宽度，否则无效。FPGA 内部加 1，时基由 DI_BASE 确定。

脉冲持续输入宽度 = (DI_WIDTH + 1) × (DI_BASE + 1) × 1us。假定 DI_BASE 取值为 0，DI_WIDTH 也取值为 0，则脉冲持续输入宽度为 1us。DI_WIDTH 和 DI_BASE 取值均为 0 ~ 999，则最大脉冲持续输入宽度为 1 秒钟。

特别注意：仅对脉冲模式有效。

3.5.3.6 设置数字输入延迟(SetDiDly)

函数原型：

```
ET_ZRC ZXUT_SetDiDly(U8 nBrdNum, U8 nDiChan, U16 nDiDly);
```

函数参数：

```
U16 nDiDly      ..... [IN] DI 延迟（10 位）
```

功能描述：

设置要延迟多长时间才可再次检测数字输入。FPGA 检测到有效的外部信号之后，必须等待该延迟时间。FPGA 否则不会再次对外部信号进行检测。

脉冲输入延迟 = DI_DLY × (DI_BASE + 1) × 1us。缺省 DI_DLY 设定为 0，则没有延迟。另若 DI_DLY 取值为 10，DI_BASE 取值为 0，则延迟时间为 10us。DI_DLY 和 DI_BASE 取值均为 0 ~ 999，则最大脉冲持续输入延迟为 1 秒钟。

特别注意：仅对脉冲模式有效。

3.5.3.7 设置数字输入时基(SetDiBase)

函数原型：

```
ET_ZRC ZXUT_SetDiBase(U8 nBrdNum, U16 nDiBase);
```

函数参数：

```
U16 nDiBase      ..... [IN] DI 时基（10 位）
```

功能描述：

指定脉冲输入延迟和宽度的时基，FPGA 内部+1。

脉冲输入时基 = (DI_BASE + 1) × 1us。DI_BASE 缺省设定为 0，则输入时基为 1us。DI_BASE 取值均为 0 ~ 999，则最大脉冲输入时基为 1 毫秒。

特别注意：（1）仅对脉冲模式有效；（2）针对所有 8 个 DI 通道有效。

3.5.4 数字输出（DO）参数接口

设定数字输出相关参数，包括使能、模式、极性、宽度及延迟等等。

关于数字输出通道参数，均遵循：

U8 nDoChan [IN] DO 通道，0 ~ 7

超出该取值范围，将得到如下返回值：

■ ZRC_INVALID_DO_CHAN，无效的 DO 通道

在 API 函数的参数描述中不再复述。

特别注意： DO 参数均针对 0#通道板（最左侧，主板），无需再指定板号。因为只有 0#通道板上的 DO 输出连接到了 IO 板上的 DO 端口。

3.5.4.1 设置数字输出使能 (SetDoEnb)

函数原型：

ET_ZRC ZXUT_SetDoEnb(U8 nDoChan, BOOL bDoEnb);

函数参数：

BOOL bDoEnb [IN] DO 使能（1 位）

功能描述：

写 1 到该接口，使能对应通道的 DO（缺省值）；写 0，则禁止该通道的 DO。只有在输出使能的情况下，才会输出指定的波形。

3.5.4.2 设置数字输出模式 (SetDoMode)

函数原型：

ET_ZRC ZXUT_SetDoMode(U8 nDoChan, U8 nDoMode);

函数参数：

U8 nDoMode [IN] DO 模式（1 位）

功能描述：

设定数字输出的模式。

- **DIO_MODE_PUL**: 脉冲模式, 可以设定脉冲延迟、宽度以及时基;
- **DIO_MODE_LEV**: 电平模式, 缺省模式, 直接由 DO_CMD 确定电平的高低。

```

////////////////////////////////////
// DIO模式

#define DIO_MODE_PUL      0      // 脉冲模式
#define DIO_MODE_LEV      1      // 电平模式

```

超出该取值范围, 将得到如下返回值:

■ **ZRC_INVALID_DIO_MODE**, 无效的 DI 模式

3.5.4.3 设置数字输出极性(SetDoPol)

函数原型:

```
ET_ZRC ZXUT_SetDoPol(U8 nDoChan, U8 nDoPol);
```

函数参数:

U8 nDoPol [IN] DO 极性 (1 位)

功能描述:

设置成功, 返回 **TRUE**; 设置失败, 则返回 **FALSE**。设定数字输出的极性。

- **POL_DEF_NEG**: 负极性, 负脉冲或低电平;
- **POL_DEF_POS**: 正极性, 正脉冲或高电平 (缺省值);

```

////////////////////////////////////
// 极性定义

#define POL_DEF_NEG      0      // 负极性
#define POL_DEF_POS      1      // 正极性

```

可能的错误返回值为:

■ **ZRC_INVALID_POL_DEF**, 无效的极性定义

3.5.4.4 设置数字输出命令(SetDoCmd)

函数原型:

```
ET_ZRC ZXUT_SetDoCmd(U8 nDoChan, BOOL bDoCmd);
```

函数参数:

BOOL bDoCmd [IN] DO 命令 (1 位)

功能描述:

不同的数字输出模式，行为不同。

- **脉冲模式**：写该接口即开始生成输出脉冲，无论写入的是 1 还是 0；
- **电平模式**：0：低电平（缺省值）；1：高电平。

3.5.4.5 设置数字输出宽度 (SetDoWidth)**函数原型:**

```
ET_ZRC ZXUT_SetDoWidth(U8 nDoChan, U16 nDoWidth);
```

函数参数:

U16 nDoWidth [IN] DO 宽度（10 位）

功能描述:

指定脉冲输出宽度，FPGA 内部加 1，时基由 DO_BASE 确定。

脉冲输出宽度 = (DO_WIDTH + 1) × (DO_BASE + 1) × 1us。假定 DO_BASE 取值为 0，DO_WIDTH 也取值为 0，则脉冲输出宽度为 1us。DO_WIDTH 和 DO_BASE 取值均为 0 ~ 999，则最大脉冲输出宽度为 1 秒钟。

特别注意：（1）仅对脉冲模式有效；（2）必须经过由 DO_DLY 指定延迟之后，才开始输出脉冲。

3.5.4.6 设置数字输出延迟 (SetDoDly)**函数原型:**

```
ET_ZRC ZXUT_SetDoDly(U8 nDoChan, U16 nDoDly);
```

函数参数:

U16 nDoDly [IN] DO 延迟（10 位）

功能描述:

指定脉冲输出延迟，时基由 DO_BASE 确定。

脉冲输出延迟 = DO_DLY × (DO_BASE + 1) × 1us。缺省 DO_DLY 设定为 0，则没有延迟。另若 DO_DLY 取值为 10，DO_BASE 取值为 0，则延迟时间为 10us。DO_DLY 和 DO_BASE 取值均为 0 ~ 999，则最大脉冲输出延迟为 1 秒钟。

特别注意：仅对脉冲模式有效。

3.5.4.7 设置数字输出时基 (SetDoBase)**函数原型:**

```
ET_ZRC ZXUT_SetDoBase(U16 nDoBase);
```

函数参数:

U16 nDoBase [IN] DO 时基（10 位）

功能描述:

指定脉冲输出延迟和宽度的时基，FPGA 内部+1。

脉冲输出时基 = (DO_BASE + 1) × 1us。DO_BASE 缺省设定为 0，则输出时基为 1us。DO_BASE 取值均为 0 ~ 999，则最大脉冲输出时基为 1 毫秒。

特别注意: (1) 仅对脉冲模式有效；(2) 针对所有 8 个 DO 通道有效。

3.6 通道参数接口函数

本章节中列举的接口函数与探伤通道有关，牵涉到各种探伤参数（比如：发射脉宽、零位偏移、TCG 等等）的设置。

通道参数中 nHardNum 和 nSoftNum 参数均遵循：

U8 nHardNum [IN] 硬通道编号，0 ~ (TOT_HARD_NUM - 1)
U8 nSoftNum [IN] 软通道编号，0 ~ 3

若参数超出上述范围，将得到如下返回值：

- ZRC_INVALID_HARD_NUM, 无效的硬通道编号
- ZRC_INVALID_SOFT_NUM, 无效的软通道编号

ZXUT-PCI 不含软通道，nSoftNum 均恒定为 0。

另外，指向核心参数的指针定义为：

ST_CORE_PARAM* pstCoreParam [IN] 指向核心参数的指针

关于核心参数 (ST_CORE_PARAM) 数据结构的定义参见后续章节。

后面的 API 函数参数描述不再复述。

3.6.1 通用通道参数接口

3.6.1.1 设置通道使能 (SetChanEnb)

函数原型:

ET_ZRC ZXUT_SetChanEnb (U8 nHardNum, U8 nSoftNum, BOOL bChanEnb);

函数参数:

BOOL bChanEnb [IN] 通道使能 (1 位)

功能描述:

设置通道使能。0 - 禁止该通道；1 - 使能该通道（缺省值）。缺省 4 个通道全部使能。

3.6.1.2 设置收发模式 (SetRtMode)

函数原型：

```
ET_ZRC ZXUT_SetRtMode (U8 nHardNum, U8 nRtMode);
```

函数参数：

```
U8 nRtMode          ..... [IN] 收发模式（1 位）
```

函数功能描述：

允许的收发模式取值为：

- 。 **RT_MODE_SOLO**：自发自收（单晶，缺省模式）；
- 。 **RT_MODE_DBL**：单发单收（双晶）。

```
//////////////////////////////////////
// 收发模式
```

```
#define RT_MODE_SOLO          0          // 自发自收(单晶)
#define RT_MODE_DBL          1          // 一发一收(双晶)
```

可能的错误返回值为：

■ **ZRC_INVALID_RT_MODE**，无效的收发模式

3.6.1.3 设置数字滤波器 (SetDigFiltr)

函数原型：

```
ET_ZRC ZXUT_SetDigFiltr (U8 nHardNum, U8 nSoftNum, U8 nDigFiltr);
```

函数参数：

```
U8 nDigFiltr          ..... [IN] 可以选择数字滤波器编号
```

函数功能描述：

用户可以选择设备实现的 FIR 数字滤波器。

■ **0**，无数字滤波器；

■ **1**，2MHz 低通滤波器（LPF）；

■ **2**，5MHz 低通滤波器（LPF）；

■ **3**，10MHz 低通滤波器（LPF）；

■ **4**，15MHz 低通滤波器（LPF）；

■ **5**，1MHz 高通滤波器（HPF）；

- 6, 2MHz 高通滤波器 (HPF);
- 7, 5MHz 高通滤波器 (HPF);
- 8, 10MHz 高通滤波器 (HPF);

- 9, 1MHz到2MHz的带通滤波器 (BPF);
- 10, 1MHz到5MHz的带通滤波器 (BPF);
- 11, 1MHz到10MHz的带通滤波器 (BPF);
- 12, 1MHz到15MHz的带通滤波器 (BPF);

- 13, 2MHz到5MHz的带通滤波器 (BPF);
- 14, 2MHz到10MHz的带通滤波器 (BPF);
- 15, 2MHz到15MHz的带通滤波器 (BPF);

- 16, 5MHz到10MHz的带通滤波器 (BPF);
- 17, 5MHz到15MHz的带通滤波器 (BPF);

- 18, 10MHz到15MHz的带通滤波器 (BPF)。

可能的错误返回值为:

- **ZRC_INVALID_DIG_FLTR**, 无效的数字滤波器;
- **ZRC_ZXUT_NOT_SUPPORT**, ZXUT-PCI 不支持数字滤波器;
- **ZRC_NO_ENOUGH_CHAN_TIME**, 没有足够的通道时间。

3.6.1.4 设置模拟滤波器 (SetAngFiltr)

函数原型:

```
ET_ZRC ZXUT_SetAngFiltr(U8 nHardNum, U8 nSoftNum, U8 nAngFiltr);
```

函数参数:

U8 nAngFiltr [IN] 可以选择模拟滤波器编号

函数功能描述:

用户可以选择设备实现的模拟滤波器。

- 0, 1 ~ 6MHz 带通滤波器 (BPF);
- 1, 3 ~ 15MHz 带通滤波器 (BPF);
- 2, 5 ~ 21MHz 带通滤波器 (BPF);
- 3, 0.5 ~ 21MHz 带通滤波器 (BPF)。

可能的错误返回值为:

- **ZRC_INVALID_ANG_FLTR**, 无效的模拟滤波器

3.6.1.5 设置检波模式 (SetDemMode)

函数原型:

```
ET_ZRC ZXUT_SetDemMode(U8 nHardNum, U8 nSoftNum, U8 nDemMode);
```

函数参数:

U8 nDemMode [IN] 检波模式 (2 位)

函数功能描述:

允许的检波模式取值为:

```
////////////////////////////////////
// 检波模式
```

```
#define DEM_MODE_FULL      0      // 全检波
#define DEM_MODE_POS      1      // 正检波
#define DEM_MODE_NEG      2      // 负检波
#define DEM_MODE_RF       3      // 射频
```

可能的错误返回值为:

■ ZRC_INVALID_DEM_MODE, 无效的检波模式

3.6.1.6 设置增益值(SetDbNum)**函数原型:**

```
ET_ZRC ZXUT_SetDbNum(U8 nHardNum, U8 nSoftNum, U16 nDbNum);
```

函数参数:

U16 nDbNum [IN] 增益值

函数功能描述:

设置增益值, 调节步进为 1, 即对应 0.1dB。最小值为 0, 即 0.0dB, 最大值可通过 ST_EXP_INFO 数据结构获得, 通常是 1100, 即 110dB。

可能的错误返回值为:

■ ZRC_INVALID_DB_NUM, 无效的增益值

3.6.1.7 设置发射脉冲宽度(SetPulWidth)**函数原型:**

```
ET_ZRC ZXUT_SetPulWidth(U8 nHardNum, U8 nSoftNum, U16 nPulWidth);
```

函数参数:

U16 nPulWidth [IN] 发射脉宽

函数功能描述:

其中 nPulWidth 的取值范围为:

```

////////////////////////////////////
// 脉冲宽度(ns)

#define PUL_WIDTH_MIN      30
#define PUL_WIDTH_MAX      2000

```

可能的错误返回值为:

■ **ZRC_INVALID_PUL_WIDTH**, 无效的发射脉宽

3.6.1.8 设置 FFT 闸门(SetFftGate)

函数原型:

```
ET_ZRC ZXUT_SetFftGate(U8 nHardNum, U8 nSoftNum, U8 nFftGate);
```

函数参数:

```
U8 nFftGate      ..... [IN] FFT 闸门 (2 位)
```

函数功能描述:

设备可以实现硬件 FFT, 用户软件可以选择硬件 FFT 依照的闸门, 设备使用选择的闸门内的回波数据进行 FFT。

```

////////////////////////////////////
// 闸门选择

#define GATE_SEL_A      0      // 闸门A
#define GATE_SEL_B      1      // 闸门B
#define GATE_SEL_C      2      // 闸门C
#define GATE_SEL_I      3      // 闸门I (界面波)

```

可能的错误返回值为:

■ **ZRC_INVALID_GATE_SEL**, 无效的闸门选择

3.6.1.9 设置回波模式(SetEchoMode)

函数原型:

```
ET_ZRC ZXUT_SetEchoMode(U8 nHardNum, U8 nSoftNum, U8 nEchoMode);
```

函数参数:

```
U8 nEchoMode      ..... [IN] 回波模式 (2 位)
```

函数功能描述:

设置通道的回波模式。该参数决定了多次采样的数据保存原则, 是最后一幅回波, 还是

某个闸门内最高的回波。

- **ECHO_MODE_FREE**: 自由模式，即取多次采样中的最后一幅波；
- **ECHO_MODE_GA**: 取闸门 A 内峰值最高的一幅波（缺省值）；
- **ECHO_MODE_GB**: 取闸门 B 内峰值最高的一幅波；
- **ECHO_MODE_GC**: 取闸门 C 内峰值最高的一幅波。

```
////////////////////////////////////
// 回波模式
```

```
#define ECHO_MODE_FREE      0      // 自由模式，取最后一幅波
#define ECHO_MODE_GA       1      // 取闸门A内最高一幅波
#define ECHO_MODE_GB       2      // 取闸门B内最高一幅波
#define ECHO_MODE_GC       3      // 取闸门C内最高一幅波
```

可能的错误返回值为：

- **ZRC_INVALID_ECHO_MODE**，无效的回波模式

3.6.1.10 设置采样深度(SetSmplDepth)

函数原型：

```
ET_ZRC ZXUT_SetSmplDepth(U8 nHardNum, U8 nSoftNum, U16 nSmplDepth,
                          U16* pnMaxSmplDepth, U32* pnSoloBrdNoAccDws);
```

函数参数：

```
U16 nSmplDepth      ..... [IN] 采样深度 (X_BW 位);
U16* pnMaxSmplDepth ..... [OUT] 允许的最大采样深度;
U32* pnSoloBrdNoAccDws ..... [OUT] 单板无累积双字数。
```

函数功能描述：

建议的采样深度范围从 512 开始，原则上可以支持不超出限制的任何数值。

- ZXUT-PCI，最大采样深度为 1536 点（1.5K）；
- ZXUT-NET，最大采样深度为 32710 点（接近 32K）。

设置采样深度，将影响通道检测范围和重复频率。

特别注意：由于将 2 次连续的 12 位采样数据合并为 1 个 32 位的采样数据进行传输，以便提高 DMA 传输效率。因此，采样深度要求必须是 2 的倍数。

可能的错误返回值为：

- **ZRC_INVALID_SMPL_DEPTH**，无效的采样深度；
- **ZRC_NO_ENOUGH_CHAN_TIME**，无足够的通道时间（pnMaxSmplDepth 返回允许的最大

采样深度)；

- **ZRC_NO_ENOUGH_BRD_MEM**，无足够的板载内存 (pnSoloBrdNoAccDws 返回单板无累积双字数)。

ZXUT 设备的板载内存的限制如下：

```
// 板内所有通道的数据量双字限制 (单次累积)
#define BRD_NOACC_MAX_DWS_PCIH7    (1536)           // 1.5K
#define BRD_NOACC_MAX_DWS_NETS4    (80 * 1024)       // 80K
```

3.6.1.11 设置零位偏移 (SetOrigDly)

函数原型：

```
ET_ZRC ZXUT_SetOrigDly(U8 nHardNum, U8 nSoftNum, S32 nOrigDly,
                       U32* pnMaxOrigDly);
```

函数参数：

```
S32 nOrigDly          ..... [IN] 零位偏移 (可正可负);
U32* pnMaxOrigDly     ..... [OUT] 允许的最大零位偏移。
```

函数功能描述：

该参数确定 PUL 脉冲与开始采样的间隔。零位偏移的取值范围为：

```
////////////////////////////////////
// 零位延迟 (ns)

#define ORIG_DLY_MIN          -20000    // -20us
#define ORIG_DLY_MAX          100000    // 100us
```

可能的错误返回值为：

- **ZRC_INVALID_ORIG_DLY**，无效的零位偏移；
- **ZRC_NO_ENOUGH_CHAN_TIME**，无足够的通道时间 (pnMaxOrigDly 返回允许的最大零位偏移)。

3.6.1.12 设置分频比 (SetFreqRatio)

函数原型：

```
ET_ZRC ZXUT_SetFreqRatio(U8 nHardNum, U8 nSoftNum, U16 nFreqRatio,
                          U16* pnMaxFreqRatio);
```

函数参数：

```
U16 nFreqRatio          ..... [IN] 分频比 (T_BW 位);
U16* pnMaxFreqRatio     ..... [OUT] 允许的最大分频比。
```

函数功能描述：

设置通道的分频比，其取值范围为：

```

////////////////////////////////////
// 分频比

#define FREQ_RATIO_MIN      1
#define FREQ_RATIO_MAX      720

```

设置分频比，将影响检测范围和重复频率。

可能的错误返回值为：

- **ZRC_INVALID_FREQ_RATIO**，无效的分频比；
- **ZRC_NO_ENOUGH_CHAN_TIME**，无足够的通道时间（pnMaxFreqRatio 返回允许的最大分频比）。

3.6.1.13 设置平均次数 (SetAvgTimes)

函数原型：

```

ET_ZRC ZXUT_SetAvgTimes(U8 nHardNum, U8 nSoftNum, U8 nAvgTimes,
                        U8* pnMaxAvgTimes);

```

函数参数：

```

U8 nAvgTimes          ..... [IN] 平均次数（4 位）；
U8* pnMaxAvgTimes     ..... [OUT] 允许的最大平均次数。

```

函数功能描述：

设置通道的平均次数，可用的平均次数设置为：

- 。 **1**：无平均（缺省值）；
- 。 **2**：2 次平均；
- 。 **4**：4 次平均；
- 。 **8**：8 次平均；
- 。 **16**：16 次平均。

可能的错误返回值为：

- **ZRC_INVALID_AVG_TIMES**，无效的平均次数；
- **ZRC_NO_ENOUGH_CHAN_TIME**，无足够的通道时间（pnMaxAvgTimes 返回允许的最大平均次数）。

3.6.1.14 设置数据开关 (SetDataSwitch)

函数原型：

```

ET_ZRC ZXUT_SetDataSwitch(U8 nHardNum, U8 nSoftNum, BOOL bSmplWave,
                          BOOL bFftWave, BOOL bCentFreq);

```

函数参数:

BOOL bSmplWave **[IN]** 采样波形使能;
BOOL bFftWave **[IN]** FFT 波形使能;
BOOL bCentFreq **[IN]** 中心频率使能。

函数功能描述:

用户可以根据实际使用情况,设置是否使能采样波形、FFT 结果波形以及中心频率数据。

可能的错误返回值为:

■ **ZRC_FFT_DISABLE**, 禁止 FFT (通道时间不够)

3.6.2 TCG 通道参数接口

设置 TCG 相关参数,包括:使能、总点数、测试点的信息等。

3.6.2.1 设置 TCG 使能(SetTcgEnb)

函数原型:

ET_ZRC ZXUT_SetTcgEnb(**U8** nHardNum, **U8** nSoftNum, **BOOL** bTcgEnb);

函数参数:

BOOL bTcgEnb **[IN]** TCG 使能 (1 位)

函数功能描述:

发送 1 表示使能 TCG, 否则关闭 TCG。

3.6.2.2 设置 TCG 总时间(SetTcgTotTime)

函数原型:

ET_ZRC ZXUT_SetTcgTotTime(**U8** nHardNum, **U8** nSoftNum, **U32** nTcgTotTime);

函数参数:

U32 nTcgTotTime **[IN]** TCG 总时间 (ns)

函数功能描述:

设置 TCG 的总时间, 时基: ns。

3.6.2.3 设置 TCG 测试点点数(SetTcgTpTot)

函数原型:

ET_ZRC ZXUT_SetTcgTpTot(**U8** nHardNum, **U8** nSoftNum, **U8** nTcgTpTot);

函数参数:

U8 nTcgTpTot **[IN]** TCG 测试点点数 (4 位)

函数功能描述:

设置 TCG 测试点点数，最多 16 个测试点。其测试点索引为 0 ~ 15。

```
#define TCG_TP_NUM_MAX    16    // 最多支持的测试点数
```

3.6.2.4 设置 TCG 测试点索引 (SetTcgTpIdx)**函数原型:**

```
ET_ZRC ZXUT_SetTcgTpIdx(U8 nHardNum, U8 nSoftNum, U8 nTcgTpIdx);
```

函数参数:

U8 nTcgTpIdx [IN] TCG 测试点索引（4 位）

函数功能描述:

设置 TCG 测试点索引，取值范围为 0 ~ (nTcgTpTot - 1)。

3.6.2.5 设置 TCG 测试点时间 (SetTcgTpTime)**函数原型:**

```
ET_ZRC ZXUT_SetTcgTpTime(U8 nHardNum, U8 nSoftNum, U32 nTcgTpTime);
```

函数参数:

U32 nTcgTpTime [IN] TCG 测试点时间（32 位，ns）

函数功能描述:

设置 TCG 测试点时间，时基为 ns。

特别注意: 在设置测试点时间之前，必须先行设置测试点索引。

3.6.2.6 设置 TCG 测试点增益 (SetTcgTpGain)**函数原型:**

```
ET_ZRC ZXUT_SetTcgTpGain(U8 nHardNum, U8 nSoftNum, U16 nTcgTpGain);
```

函数参数:

U16 nTcgTpGain [IN] TCG 测试点增益（11 位）

函数功能描述:

设置 TCG 测试点增益，其取值范围与仪器的增益相同。

特别注意: 在设置测试点增益之前，必须先行设置测试点索引。

3.6.3 闸门参数接口

本部分闸门参数接口的标法，分别指的是 4 个闸门的地址，依次为闸门 A（**GA**）、闸门 B（**GB**）、闸门 C（**GC**）和闸门 I（**GI**）。

```

////////////////////////////////////
// 闸门选择

#define GATE_SEL_A      0      // 闸门A
#define GATE_SEL_B      1      // 闸门B
#define GATE_SEL_C      2      // 闸门C
#define GATE_SEL_I      3      // 闸门I (界面波)

```

闸门参数中 nGateSel 参数均遵循：

U8 nGateSel [IN] 闸门索引，0 ~ 3

超出该取值范围，将得到如下返回值：

■ ZRC_INVALID_GATE_SEL，无效的闸门选择

后面的 API 函数参数描述不再复述。

3.6.3.1 设置闸门起点 (SetGateStart)

函数原型：

```

ET_ZRC ZXUT_SetGateStart(U8 nHardNum, U8 nSoftNum, U8 nGateSel,
                          U16 nGateStart);

```

函数参数：

U16 nGateStart [IN] 闸门起点 (X_BW 位)

函数功能描述：

设置探伤闸门的起点。

特别注意：一定要保证闸门起点确定的水平闸门位置位于采样深度之内。若设定采样深度为 512，则整个闸门位于 0 ~ 511 之内。

3.6.3.2 设置闸门宽度 (SetGateWidth)

函数原型：

```

ET_ZRC ZXUT_SetGateWidth(U8 nHardNum, U8 nSoftNum, U8 nGateSel,
                          U16 nGateWidth);

```

函数参数：

U16 nGateWidth [IN] 闸门宽度 (X_BW 位)

函数功能描述：

设置探伤闸门的宽度。

特别注意：一定要保证闸门起点确定的水平闸门位置位于采样深度之内。若设定采样深

度为 512，则整个闸门位于 0 ~ 511 之内。

3.6.3.3 设置闸门高度 (SetGateHeight)

函数原型：

```
ET_ZRC ZXUT_SetGateHeight(U8 nHardNum, U8 nSoftNum, U8 nGateSel,
                           U16 nGateHeight);
```

函数参数：

U16 nGateHeight [IN] 闸门高度 (Y_BW 位)

函数功能描述：

设置探伤闸门的高度。

特别注意：一定要保证闸门高度确定的垂直闸门位置位于 0 ~ 4095 (ADC 为 12 位) 或 0 ~ 255 (ADC 为 8 位) 之内。

3.6.3.4 设置闸门极性 (SetGatePol)

函数原型：

```
ET_ZRC ZXUT_SetGatePol(U8 nHardNum, U8 nSoftNum, U8 nGateSel,
                       U8 nGatePol);
```

函数参数：

U8 nGatePol [IN] 闸门极性 (1 位)

函数功能描述：

设置通道的闸门极性。

- 。POL_DEF_NEG：取闸门内波谷；
- 。POL_DEF_POS：取闸门内波峰（缺省值）。

```
//////////////////////////////////////
// 极性定义
```

```
#define POL_DEF_NEG                   0       // 负极性
#define POL_DEF_POS                   1       // 正极性
```

可能的错误返回值为：

■ ZRC_INVALID_POL_DEF，无效的极性定义

特别注意：仅当检波模式为射频 (**RF**) 时，才区分波峰与波谷。对于别的检波模式 (**POS**、**NEG** 和 **FULL**)，全部由 FPGA 自动设定为波峰。

3.6.4 界面波闸门跟踪

探伤设备可以硬件实现界面波闸门跟踪。

3.6.4.1 设置跟踪使能(SetTraceEnb)

函数原型:

```
ET_ZRC ZXUT_SetTraceEnb(U8 nHardNum, U8 nSoftNum, BOOL bTraceEnb);
```

函数参数:

```
BOOL bTraceEnb      .... [IN] 跟踪使能 (1 位)
```

函数功能描述:

设置界面波闸门跟踪使能 (1) 或禁止 (0)。

3.6.4.2 设置跟踪类型(SetTraceType)

函数原型:

```
ET_ZRC ZXUT_SetTraceType(U8 nHardNum, U8 nSoftNum, U8 nTraceType);
```

函数参数:

```
U8 nTraceType      .... [IN] 跟踪类型 (1 位)
```

函数功能描述:

设置界面波闸门跟踪类型。

```
//////////////////////////////////////
// 跟踪类型

#define TRACE_TYPE_PEDGE      0      // 前沿跟踪
#define TRACE_TYPE_NEDGE     1      // 后沿跟踪
```

可能的错误返回值为:

■ ZRC_INVALID_TRACE_TYPE, 无效的跟踪类型

3.6.4.3 设置跟踪点(SetTracePnt)

函数原型:

```
ET_ZRC ZXUT_SetTracePnt(U8 nHardNum, U8 nSoftNum, U16 nTracePnt);
```

函数参数:

```
U16 nTracePnt      .... [IN] 跟踪点 (X_BW 位)
```

函数功能描述:

设置界面波闸门跟踪点, 必须位于采样深度之内。

可能的错误返回值为:

■ **ZRC_INVALID_TRACE_PNT**, 无效的跟踪点

3.7 封装接口函数

下列接口函数是出于封装具体技术实现细节从而简化用户设计的目的而设计。一切与用户无关而是探伤板卡技术实现细节的部分封装于这些函数中。

3.7.1 获取 FFT 频率 (GetFftFreq, MHz)

函数原型:

```
F32 ZXUT_GetFftFreq(U16 nSmplIdx, U16 nFreqRatio);
```

函数参数:

```
U16 nSmplIdx          ..... [IN] 样本索引
U16 nFreqRatio ..... [IN] 分频比 (T_BW 位)
```

函数功能描述:

返回指定分频比下样本索引的频率值 (MHz)。

```
return (F32)(100.0 * nSmplIdx / 1024.0 / nFreqRatio);
```

3.7.2 获取检测时间 (GetTstTime, ns)

函数原型:

```
U32 ZXUT_GetTstTime(U16 nFreqRatio, U16 xCoord, U16 tCoord);
```

函数参数:

```
U16 nFreqRatio      ..... [IN] 分频比 (T_BW 位)
U16 xCoord          ..... [IN] X 坐标 (X_BW 位)
U16 tCoord          ..... [IN] T 坐标 (T_BW 位)
```

函数功能描述:

返回指定分频比、X/T 坐标下某点的检测时间 (ns)。

```
return (xCoord * nFreqRatio + tCoord) * 10;
```

3.7.3 获取检测距离 (GetTstRange, mm)

函数原型:

```
F32 ZXUT_GetTstRange(U16 nFreqRatio, U16 xCoord, U16 tCoord,
                     U32 nSoundVelo);
```

函数参数:

U16 nFreqRatio [IN] 分频比 (T_BW 位)
U16 xCoord [IN] X 坐标 (X_BW 位)
U16 tCoord [IN] T 坐标 (T_BW 位)
U32 nSoundVelo [IN] 材料声速 (m/s)

函数功能描述:

返回指定分频比、X/T 坐标下某点的检测距离 (mm)。依照最基本的物理声学公式:

$$S = \frac{V \times T}{2}$$

其中 S 表示距离, V 表示声速, T 表示飞行时间。除以 2, 表示来回往返。

```
// ns -> us  
F32 time = ZXUT_GetTstTime(nFreqRatio, xCoord, tCoord) * 0.001f;  
F32 velo = nSoundVelo * 0.001f; // mm/us  
return velo * time / 2.0f;      // mm, 已考虑往返
```

3.7.4 获取最大重复频率 (GetMaxRptFreq)

函数原型:

U32 ZXUT_GetMaxRptFreq(void);

函数参数:

无。

函数功能描述:

根据当前的探伤参数, 计算得到最大重复频率。

3.7.5 获取单块板卡无累积的采集双字数 (GetSoloBrdNoAccDaqDws)

函数原型:

U32 ZXUT_GetSoloBrdNoAccDaqDws(U8 nBrdNum);

函数参数:

略去。

函数功能描述:

获取单块板卡无累积的采集双字数。以此来确认当前的参数设置是否满足单块板卡的采集数据量的限制。

```
// 板内所有通道的数据量双字限制 (单次累积) 81,920 双字  
#define BRD_NOACC_MAX_DWS (80 * 1024)
```

```
// 板内所有通道的数据量双字限制(多次累积) 81,920 x 100 = 8,192,000双字
#define BRD_ACC_MAX_DWS (BRD_NOACC_MAX_DWS * MAX_ACC_TIMES)

// 转换为字节8,192,000双字x 4 = 32,768,000字节= 31.25MB
#define BRD_ACC_MAX_BYTES (BRD_ACC_MAX_DWS * 4)
```

3.7.6 初始化 FFT (FftInit)

函数原型:

```
void ZXUT_FftInit(S32 nFftNum);
```

函数参数:

S32 nFftNum [IN] FFT 点数, 必须是 2 的幂, 比如 512、1024、2048 等

函数功能描述:

完成 FFT 的初始化工作, 指定 FFT 点数。

3.7.7 执行 FFT (FftExec)

函数原型:

```
void ZXUT_FftExec(ST_COMP_NUM* pstFftData);
```

函数参数:

ST_COMP_NUM* pstFftData [IN/OUT] FFT 输入/输出数据

函数功能描述:

将需要执行 FFT 的数据置于指针 pstFftData 指向的内存空间, 完成后输出数据也存于该存储空间。

```
////////////////////////////////////
// 复数

typedef struct _ST_COMP_NUM
{
    double real;    // 实部
    double image;   // 虚部
} ST_COMP_NUM;
```

特别注意: ZXUT-PCI 无硬件 FPGA 功能, 需要软件计算 FFT。

3.7.8 初始化实时存储 (InitRtMem)

函数原型:

```
ET_ZRC ZXUT_InitRtMem(U8 nHardNum, U8 nSoftNum, BOOL nRtMemEnb,  
    U32 nRtMemTime, char* strRtMemFileName, U32* pnRtMemTimes,  
    U64* pnRtMemFileSize);
```

函数参数:

BOOL nRtMemEnb		[IN] 实时存储使能
U32 nRtMemTime		[IN] 实时存储时间（秒数）
char* strRtMemFileName		[IN] 实时存储文件名
U32* pnRtMemTimes		[OUT] 实时存储次数
U64* pnRtMemFileSize		[OUT] 实时存储文件大小（字节数）

函数功能描述:

初始化实时存储，设置实时存储参数。

具体 API 函数使用，参见演示软件使用说明。

第 4 章 ZXUT-NET 接口函数使用范例

本章将一步一步地讲述如何使用第 3 章所描述的 API 接口函数来对 ZXUT 探伤设备进行设置，同时从原理的角度讲述探伤参数的实际意义。

4.1 核心参数 (ST_CORE_PARAM) 的数据结构描述

```

////////////////////////////////////
// 核心参数定义

typedef struct _ST_CORE_PARAM
{
    U8      m_nBrdSync;           // 板间同步
    U32     m_nRptFreq;           // 重复频率

    U32     m_anSynDly            [MAX_BRD_NUM]; // 同步延迟

    BOOL    m_abChanEnb          [TOT_HARD_NUM][MAX_SOFT_NUM]; // 通道使能
    U32     m_anChanShare        [TOT_HARD_NUM][MAX_SOFT_NUM]; // 通道时间占额
    U16     m_anSmplDepth        [TOT_HARD_NUM][MAX_SOFT_NUM]; // 采样深度
    U16     m_anOrigDly          [TOT_HARD_NUM][MAX_SOFT_NUM]; // 零位延迟
    U16     m_anFreqRatio        [TOT_HARD_NUM][MAX_SOFT_NUM]; // 分频比
    U8      m_anAvgTimes         [TOT_HARD_NUM][MAX_SOFT_NUM]; // 平均次数
    BOOL    m_abSmplWave         [TOT_HARD_NUM][MAX_SOFT_NUM]; // 采样波形
    BOOL    m_abFftWave          [TOT_HARD_NUM][MAX_SOFT_NUM]; // FFT波形
    BOOL    m_abCentFreq         [TOT_HARD_NUM][MAX_SOFT_NUM]; // 中心频率
    U8      m_anDigFltr          [TOT_HARD_NUM][MAX_SOFT_NUM]; // 数字滤波
} ST_CORE_PARAM;

```

下面对几个关键参数进行描述。

4.1.1 逻辑通道 (LogChan)

在某些应用场合，需要使用同一个硬件通道却只是某些参数（比如增益、零偏等）不同而构建软件通道。ZXUT-NET 板卡含有 4 个硬件通道，而每个硬件通道包含 4 个软件通道。从逻辑上来说，每个板卡可以构建 $4 \times 4 = 16$ 个通道。

```

#define MAX_BRD_NUM      10           // 支持的最多通道板卡数

#define MAX_HARD_NUM     4            // 板内最大硬通道数
#define MAX_SOFT_NUM     4            // 每个硬通道包含的最大软通道数

#define TOT_HARD_NUM     (MAX_BRD_NUM * MAX_HARD_NUM) // 硬通道总数

```

```
#define TOT_LOG_NUM      (TOT_HARD_NUM * MAX_SOFT_NUM)    // 逻辑通道数
```

而对于大多数的用户而言，并不存在使用软件通道的应用场合。此时，需要将 2.6 节描述的 nSoftNum 设定为 0 即可。

4.1.2 采样深度和分频比 (SmplDepth & FreqRatio)

ZXUT 的数据采样率 (f_s) 为 100MHz，即采样周期为 10ns。任何的数据采样，均是从超声波激励脉冲的上升沿开始的。连续的采样点数，称为采样深度 (**SMPL_DEPTH**)，比如设定为 512 点。那么总的采样时间为：

$$512 \times 10\text{ns} = 5120\text{ns} = 5.12\mu\text{s}$$

钢纵波的声速为 5940m/s，即 5.94mm/us。那么 5.12us 传播的距离为：

$$5.12\mu\text{s} \times 5.94\text{mm}/\mu\text{s} = 30.4128\text{mm}$$

考虑超声波的往返，实际的超声波传播的距离为：

$$30.4128\text{mm} / 2 = 15.2064\text{mm}$$

这个超声波的传播，实际上就是超声波的检测范围。15mm 的检测范围是非常小的，为了增大检测范围，可以单纯地增加 SMPL_DEPTH，但该参数毕竟有限。另外的方式，就是引入分频比 (**FREQ_RATIO**) 的概念。该参数从 1 开始取值，表示没有分频。这样采样周期变为：

$$10\text{ns} \times \text{FREQ_RATIO}$$

比如，设定 FREQ_RATIO = 10，则采样周期为 $10\text{ns} \times 10 = 100\text{ns}$ 。同样 512 点的 SMPL_DEPTH 可以表示的检测范围变为约 150mm。

4.1.3 数据开关 (DataSwitch)

由以下数据成员构建：

```
BOOL    m_abSmplWave   [TOT_HARD_NUM] [MAX_SOFT_NUM];    // 采样波形
BOOL    m_abFftWave    [TOT_HARD_NUM] [MAX_SOFT_NUM];    // FFT波形
BOOL    m_abCentFreq   [TOT_HARD_NUM] [MAX_SOFT_NUM];    // 中心频率
```

4.1.3.1 采样波形 (SmplWave)

在某些应用场合，仅需要闸门特征数据，不需要采样波形，可以将该位设置为 0，采样波形不再包含在采样缓冲区中。可以减少网络数据量，提高重复频率。

4.1.3.2 FFT 波形 (FftWave)

如果重复频率允许的话，可以使能 FFT，这样可以极大地减少 CPU 的计算时间。但可能会降低重复频率，同时网络数据量也会增加。

4.1.3.3 中心频率(CentFreq)

在某些应用场合，不需要 FFT 波形，但需要 FFT 曲线中的中心频率。该频率可有左-3dB 和右-3dB 的频率点求均值获得。

4.1.4 通道时间&同步延迟占额

假定使能了 4 个通道，分别为 0.0、1.1、2.2 和 3.3，其通道时间占额均使用缺省值 1；又假定根据重复频率 100Hz 得到的重复周期为 10ms，那么这四个通道的通道时间均相同为 $10\text{ms} / (1 + 1 + 1 + 1) = 10\text{ms} / 4 = 2.5\text{ms}$ 。

现在设置四个通道的通道时间占额分别为 1、2、3 和 4，那么每个通道时间份额为： $10\text{ms} / (1 + 2 + 3 + 4) = 1\text{ms}$ ，而且：

- 通道 0.0 的通道时间为 $1\text{ms} \times 1 = 1\text{ms}$ ，占 $1/10$ ；
- 通道 1.1 的通道时间为 $1\text{ms} \times 2 = 2\text{ms}$ ，占 $2/10 = 1/5$ ；
- 通道 2.2 的通道时间为 $1\text{ms} \times 3 = 3\text{ms}$ ，占 $3/10$ ；
- 通道 3.3 的通道时间为 $1\text{ms} \times 4 = 4\text{ms}$ ，占 $4/10 = 2/5$ 。

通过这种指定通道时间占额的方式，可以设置各个通道如何瓜分重复时间。这在不同的通道，其检测范围极大不同的情况下，指定通道时间尤为有用。

4.1.4.1 铁轨检测的设置范例

现在加入同步延迟的设置，情况稍微复杂一些。同步延迟指的是每块板卡的第一个通道距离同步头的延迟时间。下图中的 5#板卡，3.3 通道的同步延迟为 2 份。

现在假定重复周期为 700us，每块板卡的总的份额（包含同步延迟和通道时间）为 7 份。

- 1#板卡：0, 1, 1, 1, 4 = 7
- 2#板卡：0, 4, 1, 1, 1 = 7
- 3#板卡：1, 1, 1, 1, 3 = 7
- 4#板卡：0, 1, 4, 1, 1 = 7
- 5#板卡：2, 1, 1, 1, 2 = 7
- 6#板卡：0, 1, 1, 4, 1 = 7
- 7#板卡：3, 1, 1, 1, 1 = 7

每一份额对应的实际时间为 $700\text{us} / 7 = 100\text{us}$ 。那么每个通道对应的同步延迟和通道时间随之可以计算。比如 6#板卡 4.3@4，表示这是 4#探轮的第三个通道对应的通道时间份额为 4，实际通道时间为 $4 \times 100\text{us} = 400\text{us}$ 。

通道延迟

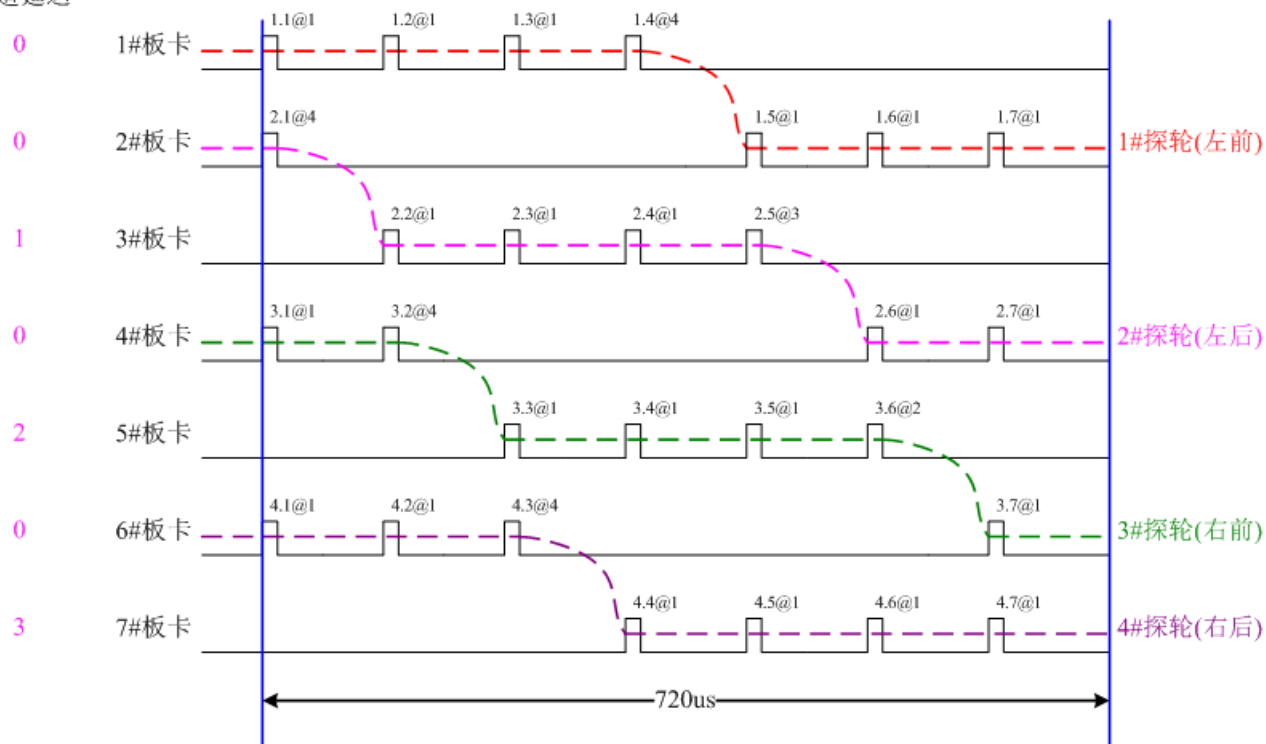


图 4-1：铁轨检测范例

特别注意：上图仅是一个范例，可能存在多种设置。用户请根据检测实际进行设置。

对应的通道使能对话框如下设置：

1#板卡				2#板卡				3#板卡				4#板卡			
1#	2#	3#	4#	5#	6#	7#	8#	9#	10#	11#	12#	13#	14#	15#	16#
☒ 1-1	☒ 2-1	☒ 3-1	☒ 4-1	☒ 5-1	☒ 6-1	☒ 7-1	☒ 8-1	☒ 9-1	☒ 10-1	☒ 11-1	☒ 12-1	☒ 13-1	☒ 14-1	☒ 15-1	☒ 16-1
☐ 1-2	☐ 2-2	☐ 3-2	☐ 4-2	☐ 5-2	☐ 6-2	☐ 7-2	☐ 8-2	☐ 9-2	☐ 10-2	☐ 11-2	☐ 12-2	☐ 13-2	☐ 14-2	☐ 15-2	☐ 16-2
☐ 1-3	☐ 2-3	☐ 3-3	☐ 4-3	☐ 5-3	☐ 6-3	☐ 7-3	☐ 8-3	☐ 9-3	☐ 10-3	☐ 11-3	☐ 12-3	☐ 13-3	☐ 14-3	☐ 15-3	☐ 16-3
☐ 1-4	☐ 2-4	☐ 3-4	☐ 4-4	☐ 5-4	☐ 6-4	☐ 7-4	☐ 8-4	☐ 9-4	☐ 10-4	☐ 11-4	☐ 12-4	☐ 13-4	☐ 14-4	☐ 15-4	☐ 16-4
1	1	1	4	4	1	1	1	1	1	1	3	1	4	1	1
全选 全清 0				全选 全清 0				全选 全清 1				全选 全清 0			

图 4-2：通道使能的设置

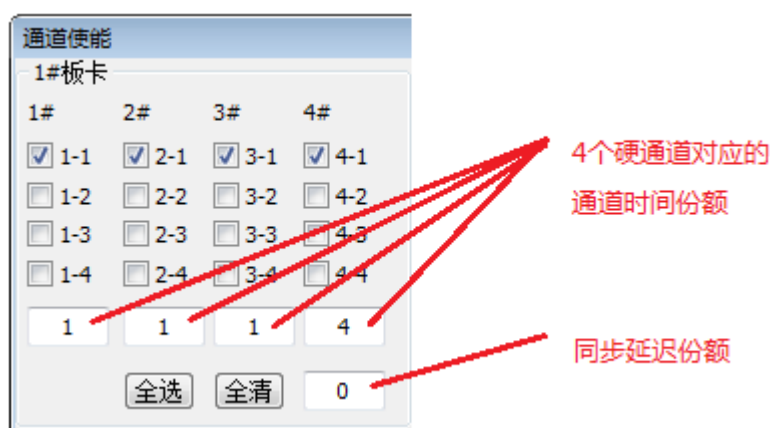


图 4-3：通道时间和同步延迟的份额的设置

4.2 中间件导出信息 (ST_EXP_INFO) 的数据结构描述

```

////////////////////////////////////
// 中间件导出信息定义

typedef struct _ST_EXP_INFO
{
    //////////////////////////////////
    // 版本信息

    U8      m_nPcbVer;    // PCB版本
    U16     m_nFpgaVer;   // FPGA版本
    U16     m_nDevVer;    // 设备版本
    U16     m_nMidVer;    // 中间件版本

    //////////////////////////////////
    // 位宽信息

    U8      m_nStBw;      // 采样次数
    U8      m_nAtBw;      // 累积次数
    U8      m_nXBw;       // 采样深度
    U8      m_nYBw;       // 采样位数
    U8      m_nTBw;       // 时间小数

    //////////////////////////////////
    // 板号

    char     m_aastrBrdSn [MAX_BRD_NUM][BRD_SN_TYPE_NUM]
                                     [BRD_SYN_TYPE_LEN * 2 + 1];

    //////////////////////////////////

```

```

// 配置信息

U8      m_nTotBrd;      // 已连接通道板卡总数
U8      m_nTotHard;     // 板内硬通道总数
U8      m_nTotSoft;     // 每个硬通道包含的软通道总数
U8      m_nInnerSync;   // 板内通道同步方式
U16     m_nMaxGain;     // 最大增益，表示dB

U8      m_anMacAddr     [MAX_BRD_NUM][6]; // 网卡MAC地址
HANDLE  m_ahDaqEvt       [MAX_BRD_NUM];    // 采集事件
ST_DAQ_BUF* m_apstDaqBuf [MAX_BRD_NUM];    // 采集缓冲区
U8      m_anBrdIdx       [MAX_BRD_NUM];    // 包含连接板卡的板号

////////////////////////////////////
// 系统时序解析结果

U32     m_nDaqRate;     // 采集率
U8      m_nSmplTimes;   // 采样次数
U8      m_nAccTimes;    // 累积次数

F32     m_afBrdNetRate [MAX_BRD_NUM];      // 每块板卡的网络速率
F32     m_fSysNetRate;   // 整个系统的网络速率
} ST_EXP_INFO;

```

这些导出信息由中间件自动收集，用户访问该数据成员即可访问。

4.2.1 版本信息 (VerInfo)

```

U8      m_nPcbVer;      // PCB版本
U16     m_nFpgaVer;     // FPGA版本
U16     m_nDevVer;      // 设备版本
U16     m_nMidVer;      // 中间件版本

```

除去 PCB 版本为 8 位之外，其余均为主版本号（高 8 位）和次版本号（低 8 位）构建的 16 位版本号。

4.2.2 位宽信息 (BwInfo)

```

U8      m_nStBw;        // 采样次数
U8      m_nAtBw;        // 累积次数
U8      m_nXBw;         // 采样深度
U8      m_nYBw;         // 采样位数
U8      m_nTBw;         // 时间小数

```

采样次数和累积次数均不会超过 100，因此 $m_nStBw = m_AtBw = 8$ 。

采样数据的 X 坐标范围为：0 ~ (SMPL_DEPTH-1)，用位数用 X_BW 来表示。在 ZXUT-NET 中，最大采样深度为 32718，因此 $X_BW = 15$ 。

数据采集所使用的 ADC 的采样精度/位数，用 Y_BW 来表示。在 ZXUT-NET， $Y_BW = 12$ 。采集数据的 Y 坐标范围为：0 ~ ($2^{(Y_BW)} - 1$)，即 0 ~ 4095。

分频比的位数用 T_BW 来表示。在 ZXUT-NET 中， $T_BW = 10$ 。

4.2.3 板号 (BrdSn)

```
char          m_aastrBrdSn  [MAX_BRD_NUM][BRD_SN_TYPE_NUM]
                                     [BRD_SYN_TYPE_LEN * 2 + 1];
```

存放 ZXUT-NET 中插入板卡的各种电路板和模块的序号。

4.2.4 配置信息

```
U8            m_nTotBrd;        // 已连接通道板卡总数
U8            m_nTotHard;       // 板内硬通道总数
U8            m_nTotSoft;       // 每个硬通道包含的软通道总数
U8            m_nInnerSync;     // 板内通道同步方式
U16           m_nMaxGain;       // 最大增益，表示dB

U8            m_anMacAddr       [MAX_BRD_NUM][6]; // 网卡MAC地址
HANDLE        m_ahDaqEvt        [MAX_BRD_NUM];    // 采集事件
ST_DAQ_BUF*   m_apstDaqBuf      [MAX_BRD_NUM];    // 采集缓冲区
U8            m_anBrdIdx        [MAX_BRD_NUM];    // 包含连接板卡的板号
```

4.2.4.1 已连接通道板卡总数 (TotBrd)

比如系统中插入 5 块板卡组建 20 通道，那么 $m_nTotBrd = 5$ 。

```
#define MAX_BRD_NUM      10        // 支持的最多通道板卡数
```

4.2.4.2 板内硬通道总数 (TotHard)

目前硬件版本的 ZXUT-NET， $m_nTotHard$ 固定为 4。

```
#define MAX_HARD_NUM     4          // 板内最大硬通道数
```

4.2.4.3 每个硬通道包含的软通道总数 (TotSoft)

目前硬件版本的 ZXUT-NET， $m_nTotSoft$ 固定为 4。

```
#define MAX_SOFT_NUM     4          // 每个硬通道包含的最大软通道数
```

4.2.4.4 板内同步关系 (InnerSync)

通道板之间的串并关系，0 表示板内硬通道间为串行关系，1 表示板内硬通道间为并行关系。

- 假定板内 4 个硬通道串行，并包含 4 个软通道，简称为 **S4.4** 配置；
- 假定板内 2 个硬通道并行，无软通道，简称为 **P2.1** 配置。

4.2.4.5 通道板模拟电路的最大增益 (MaxGain)

目前 ZXUT-NET 均为 110dB，即 $m_nMaxGain = 1100$ 。

4.2.4.6 通道板网卡 MAC 地址 (MacAddr)

若板卡 MAC 地址为：8C-DC-D4-43-9D-BF，则 $m_anMacAddr[0] = 0x8C$ ， $m_anMacAddr[1] = 0xDC$ ，以此类推。用户可能需要该地址进行软件加密。

特别重要：多个探伤板卡组建的超声波探伤系统，MAC 地址不能重复。否则会出现网络通讯异常的问题。

4.2.4.7 采集缓冲区地址 (DaqBuf)

返回采集缓冲区地址，用以访问数据结构成员。

4.2.4.8 采集事件句柄地址 (DaqEvt)

该函数返回采集时间句柄地址，用以访问采集事件。该采集事件表明采集数据已被正确接收，且已经按照 **ST_DAQ_BUF** 的数据结构进行了解析。

4.2.5 系统时序解析结果

```

U32      m_nDaqRate;      // 采集率
U8       m_nSmplTimes;    // 采样次数
U8       m_nAccTimes;     // 累积次数

F32      m_afBrdNetRate[MAX_BRD_NUM];      // 每块板卡的网络速率
F32      m_fSysNetRate;    // 整个系统的网络速率

```

对于高重复频率的应用，比如达到 kHz 级别。为了降低数据采集率 (**DAQ_RATE**)，可将采样数据在 ZXUT-NET 中累积，之后再一次性地传递给服务器。这就是累积次数 (**ACC_TIMES**) 的概念。增加累积，降低了采集率，但增加了单次采集的数据量。

单次累积的数据量称为采集长度 (**DAQ_LEN**)。

当客户端网络传输率 (**NET_RATE**) 高于 40MB/s 时，加入采样次数 (**SMPL_TIMES**) 的概念。假设采样次数设定为 3，则表示超声波每发射/接收 3 次，将会保留探伤闸门内波幅最高的那一次。这样就降低了采集率，同时也降低了网络传输率。

几个参数之间的关系为：

$$\text{DAQ_RATE} = \text{RPT_FREQ} / (\text{SMPL_TIMES} \times \text{ACC_TIMES})$$
$$\text{NET_RATE} = \text{DAQ_RATE} \times \text{ACC_TIMES} \times \text{DAQ_LEN}$$

通过调用 ZXUT_ResolveSysTiming 函数可以获得当前重复频率下的上述参数。

4.3 探伤参数的设置步骤

依照如下的步骤对探伤参数进行设置，这些步骤代码位于：

```
BOOL CZxutApp::InitDevice(void)
```

函数中。凡是标注***的步骤，是必需的，不能缺少。

4.3.1 *** 打开探伤设备 (OpenDevice)

根据探伤系统中：（1）已插入板卡的块数；（2）探伤设备的首 IP 地址；（3）网络通讯端口，来打开探伤设备。

```
////////////////////////////////////  
//  
// 打开探伤设备  
//  
////////////////////////////////////  
  
BOOL CZxutApp::OpenDevice(void)  
{  
    ET_ZRC ret_code;  
    m_nClientIpAddr =  
        (m_stFlawParam.m_anIpAddr[0] << 3 * 8) +  
        (m_stFlawParam.m_anIpAddr[1] << 2 * 8) +  
        (m_stFlawParam.m_anIpAddr[2] << 1 * 8) +  
        (m_stFlawParam.m_anIpAddr[3] << 0 * 8);  
  
    // 打开PCI探伤设备  
    ret_code = ZXUT_OpenDevice(ZXUT_TYPE_PCI, 0, 0, &m_pstExpInfo);  
    if (ret_code == ZRC_API_RET_SUCCESS)  
    {  
        m_nZxutType = ZXUT_TYPE_PCI;  
    }  
    else  
    {  
        // 已侦测到PCI板卡，但返回值错误
```

```
if (ret_code != ZRC_NO_BRD_INSERT)
{
    switch (ret_code)
    {
        case ZRC_SYS_INFO_MISMATCH:
            DispMsg(ZXUT_MSG_INFO, "反馈的系统信息不全相同");
            return FALSE;

        case ZRC_INVALID_PCB_VER:
            DispMsg(ZXUT_MSG_INFO, "无效的PCB版本");
            return FALSE;

        case ZRC_INVALID_FPGA_VER:
            DispMsg(ZXUT_MSG_INFO, "无效的FPGA版本");
            return FALSE;

        case ZRC_INVALID_TOT_HARD:
            DispMsg(ZXUT_MSG_INFO, "无效的硬通道总数");
            return FALSE;

        case ZRC_INVALID_TOT_SOFT:
            DispMsg(ZXUT_MSG_INFO, "无效的软通道总数");
            return FALSE;

        default:
            DispMsg(ZXUT_MSG_ERR, "未知错误%d", ret_code);
            return FALSE;
    }
}

// PCI通道板没有检测到, 马上连接网络
ret_code = OpenNetDev();
switch (ret_code)
{
    case ZRC_API_RET_SUCCESS:
        break;

    case ZRC_CREATE_SOCKET_FAIL:
        DispMsg(ZXUT_MSG_ERR, "创建TCP套接字失败");
        return FALSE;

    case ZRC_BIND_SERVER_FAIL:
        DispMsg(ZXUT_MSG_ERR, "绑定服务器地址和端口失败");
        return FALSE;
```



```
case ZRC_LISTEN_SERVER_FAIL:
    DispMsg(ZXUT_MSG_ERR, "服务器监听失败");
    return FALSE;

case ZRC_USR_CANCEL_CONNECT:
    DispMsg(ZXUT_MSG_INFO, "已取消客户端连接, 退出系统\n");
    ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
    return FALSE;

case ZRC_NO_CLIENT_CONNECT:
    DispMsg(ZXUT_MSG_INFO, "一个客户端都没有连接, 退出系统\n");
    ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
    return FALSE;

case ZRC_NO_MAST_BRD:
    DispMsg(ZXUT_MSG_INFO, "没有插入主通道板\n");
    ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
    return FALSE;

default:
    DispMsg(ZXUT_MSG_ERR, "等待客户端连接出现未知错误%d", ret_code);
    return FALSE;
}

ret_code = ZXUT_OpenDevice(ZXUT_TYPE_NET, m_nTotBrd,
    m_sockZxut, &m_pstExpInfo);
switch (ret_code)
{
case ZRC_API_RET_SUCCESS:
    m_nZxutType = ZXUT_TYPE_NET;
    break;

case ZRC_SYS_INFO_MISMATCH:
    DispMsg(ZXUT_MSG_INFO, "反馈的系统信息不全相同");
    ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
    return FALSE;

case ZRC_INVALID_PCB_VER:
    DispMsg(ZXUT_MSG_INFO, "无效的PCB版本");
    ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
    return FALSE;

case ZRC_INVALID_FPGA_VER:
```

```

        DispMsg(ZXUT_MSG_INFO, "无效的FPGA版本");
        ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
        return FALSE;

    case ZRC_INVALID_TOT_HARD:
        DispMsg(ZXUT_MSG_INFO, "无效的硬通道总数");
        ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
        return FALSE;

    case ZRC_INVALID_TOT_SOFT:
        DispMsg(ZXUT_MSG_INFO, "无效的软通道总数");
        ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
        return FALSE;

    default:
        DispMsg(ZXUT_MSG_ERR, "打开网络探伤设备出现未知错误%d", ret_code);
        ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
        return FALSE;
    }
}

////////////////////////////////////
// 确认设备版本号是否匹配

#if 1

    if (m_pstExpInfo->m_nDevVer != ZXUT_DEV_VER)
    {
        DispMsg(ZXUT_MSG_ERR, "反馈的设备版本号不匹配，
            需要V%d.%d，实际: V%d.%d\n",
            ZXUT_DEV_VER / 256, ZXUT_DEV_VER % 256,
            m_pstExpInfo->m_nDevVer / 256, m_pstExpInfo->m_nDevVer % 256);
        ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
        return FALSE;
    }

#endif

////////////////////////////////////
// 确认中间件版本号是否匹配

#if 1

    if (m_pstExpInfo->m_nMidVer != ZXUT_MID_VER)

```

```

{
    DispMsg(ZXUT_MSG_ERR, "反馈的中间件版本号不匹配, 需要V%d.%d, 实际:
V%d.%d\n",
            ZXUT_MID_VER / 256, ZXUT_MID_VER % 256,
            m_pstExpInfo->m_nMidVer / 256, m_pstExpInfo->m_nMidVer % 256);
    ZXUT_CloseDevice(APP_EXIT_CMD_RETRY);
    return FALSE;
}

#endif

////////////////////////////////////
// 依照设备类型, 设置设备功能标志

m_bPci = (m_nZxutType == ZXUT_TYPE_PCI); // PCI设备
m_bPciH7 = m_bPci & (m_pstExpInfo->m_nInnerSync == BRD_SYNC_SERI) &
(m_pstExpInfo->m_nPcbVer == 7);

m_bNet = (m_nZxutType == ZXUT_TYPE_NET); // NET设备
m_bNetP2 = m_bNet & (m_pstExpInfo->m_nInnerSync == BRD_SYNC_PARA);
m_bNetS4 = m_bNet & (m_pstExpInfo->m_nInnerSync == BRD_SYNC_SERI);

if (m_bNetP2)
{
    m_bDigFltrEnb = TRUE;
    m_bAngFltrEnb = TRUE;
    m_bImpModeEnb = TRUE;
    m_bBrdSyncEnb = FALSE;
    m_bDevUpdEnb = TRUE;
    strcpy(m_strZxutType, "NET-P2");
    BRD_NOACC_MAX_DWS = 0;
}
else if (m_bNetS4)
{
    m_bDigFltrEnb = TRUE;
    m_bAngFltrEnb = FALSE;
    m_bImpModeEnb = FALSE;
    m_bBrdSyncEnb = TRUE;
    m_bDevUpdEnb = TRUE;
    strcpy(m_strZxutType, "NET-S4");
    BRD_NOACC_MAX_DWS = BRD_NOACC_MAX_DWS_NETS4;
}
else if (m_bPciH7)
{

```

```
m_bDigFltrEnb = FALSE;
m_bAngFltrEnb = TRUE;
m_bImpModeEnb = FALSE;
m_bBrdSyncEnb = FALSE;
m_bDevUpdEnb = FALSE;
strcpy(m_strZxutType, "PCI-S4");
BRD_NOACC_MAX_DWS = BRD_NOACC_MAX_DWS_PCIH7;
}

////////////////////////////////////
// 设置Y方向参数

U8 y_bw = m_pstExpInfo->m_nYBw; // 12
m_nSmplWaveHeight = (1 << y_bw); // 4096
// 4096 / 512 = 8
m_nSmplWaveYRatio = m_nSmplWaveHeight / SHOW_SMPL_HEIGHT;
// 自动增益高度: 80%
m_nAutoGainHeight = (U16) (0.8f * ((1 << y_bw) - 1));

return TRUE;
}
```



图 4-4：等待客户端连接对话框

若指定数目的客户端已经连接，软件将自动关闭该对话框。否则，对话框一直出现，知道用户按下“进入系统”按钮，或关闭对话框退出。

4.3.2 设置数字输入（DI）参数

此步骤可选，不是必需的。

```
////////////////////////////////////  
//  
// 设置DI参数  
//  
////////////////////////////////////  
  
void CZxutApp::SetDiParam(void)  
{  
    ST_DIG_IN& stDigIn = m_stFlawParam.m_stDigIn;  
  
    U32 di_base = (stDigIn.m_nBase + 1) * 100 - 1;
```

```

ZXUT_SetDiBase(0, di_base);

ZXUT_SetDiTstEnb (0, stDigIn.m_bTstEnb);
ZXUT_SetDiTstFreq (0, stDigIn.m_nTstFreq);
ZXUT_SetDiTstWidth (0, stDigIn.m_nTstWidth);
ZXUT_SetDiTstChan (0, stDigIn.m_nTstChan);

for (U8 i = 0; i < TOT_DI_NUM; i++)
{
    ZXUT_SetDiEnb (0, i, stDigIn.m_bEnb [i]);
    ZXUT_SetDiMode(0, i, stDigIn.m_nMode [i]);
    ZXUT_SetDiPol (0, i, stDigIn.m_nPol [i]);
    ZXUT_SetDiWidth(0, i, stDigIn.m_nWidth [i]);
    ZXUT_SetDiDly (0, i, stDigIn.m_nDly [i]);
}
}

```

4.3.3 设置数字输出（DO）参数

此步骤可选，不是必需的。

```

////////////////////////////////////
//
// 设置DO参数
//
////////////////////////////////////

void CZxutApp::SetDoParam(void)
{
    ST_DIG_OUT& stDigOut = m_stFlawParam.m_stDigOut;

    U32 do_base = (stDigOut.m_nBase + 1) * 100 - 1;
    ZXUT_SetDoBase(do_base);

    for (U8 i = 0; i < TOT_DO_NUM; i++)
    {
        ZXUT_SetDoEnb(i, stDigOut.m_bEnb[i]);
        ZXUT_SetDoMode(i, stDigOut.m_nMode[i]);
        ZXUT_SetDoPol(i, stDigOut.m_nPol[i]);
        ZXUT_SetDoCmd(i, 0);
        ZXUT_SetDoWidth(i, stDigOut.m_nWidth[i]);
        ZXUT_SetDoDly(i, stDigOut.m_nDly[i]);
    }
}
}

```

4.3.4 设置编码器（ENC）参数

此步骤可选，不是必需的。

```
////////////////////////////////////  
//  
// 设置ENC参数  
//  
////////////////////////////////////  
  
void CZxutApp::SetEncParam(void)  
{  
    for (U8 brd_num = 0; brd_num < m_nTotBrd; brd_num++)  
    {  
        ZXUT_SetEncCntr(brd_num, 0, 0); // 初始化编码器  
        ZXUT_SetEncCntr(brd_num, 1, 0); // 初始化编码器  
        ZXUT_SetEncCntr(brd_num, 2, 0); // 初始化编码器  
        ZXUT_SetEncCntr(brd_num, 3, 0); // 初始化编码器  
  
        ST_ENC_MODU& enc_modu = m_stFlawParam.m_stEncModu[brd_num];  
  
        ZXUT_SetEncMode(brd_num, enc_modu.m_nMode);  
        ZXUT_SetEncTstFreq(brd_num, enc_modu.m_nTstFreq);  
  
        UpdEncFltr(brd_num, enc_modu.m_nEncFltr);  
  
        for (U8 j = 0; j < TOT_ENC_NUM; j++)  
        {  
            if (enc_modu.m_bScan[j])  
            {  
                ZXUT_SetScanEnc(brd_num, j);  
            }  
        }  
  
        for (U8 j = 0; j < TOT_ENC_NUM; j++)  
        {  
            ZXUT_SetEncType (brd_num, j, enc_modu.m_nType[j]);  
            ZXUT_SetEncEnb (brd_num, j, enc_modu.m_bEnb[j]);  
            ZXUT_SetEncPol (brd_num, j, enc_modu.m_nPol[j]);  
        }  
    }  
}
```

4.3.5 *** 设置探伤参数 (SetFlawParam)

请参见代码函数 `void CZxutApp::SetFlawParam(void)`。

4.3.6 *** 设置重复频率 (SetRptFreq)

```
////////////////////////////////////  
// 设置重复频率
```

```
void CZxutApp::SetRptFreq(void)  
{  
    ZXUT_SetRptFreq(&m_stFlawParam.m_stCoreParam);  
    ResolveSysTiming(); // 解析系统时序  
}
```

4.3.7 *** 创建逻辑通道表

```
CreateLogChanTab(); // [7] 创建逻辑通道表  
m_nLogIdx = 0;  
SetCrrntChan();
```

4.3.8 *** 创建数据处理线程

用户必须创建数据处理线程，用于探伤数据的显示等等。

```
////////////////////////////////////  
//  
// 创建数据处理线程  
//  
////////////////////////////////////
```

```
void CZxutApp::CreateDataProcThrd(void)  
{  
    for (U8 i = 0; i < m_nTotBrd; i++)  
    {  
        if (m_pDataProcThrd[i])  
        {  
            continue;  
        }  
  
        m_pDataProcThrd[i] = new CDataProcThrd(i);  
        if (m_pDataProcThrd[i])  
        {  
            m_pDataProcThrd[i]->Start((LPVOID) this);  
        }  
    }  
}
```



```

        m_pDataProcThrd[i]->ResumeThrd();
    }
}
}

```

4.4 探伤数据结构

这些探伤数据结构，定义于 ZxutCmn.h 头文件。

4.4.1 距离补偿（TCG）参数

存储距离补偿点的距离、时间和增益信息。

```

////////////////////////////////////
// TCG (距离补偿) 参数

#define TCG_TP_NUM_MAX    16 // 最多支持的测试点数

typedef struct _ST_TCG_PARAM
{
    U8      m_bTcgEnb;      // 使能
    U8      m_nTcgTpTot;    // 总测试点数

    F32     m_fTcgTpDist[TCG_TP_NUM_MAX]; // 测试点距离, mm
    U32     m_nTcgTpTime[TCG_TP_NUM_MAX]; // 测试点时间, ns
    U16     m_nTcgTpGain[TCG_TP_NUM_MAX]; // 测试点增益
} ST_TCG_PARAM;

```

4.4.2 闸门判伤特征值（结果）

ZXUT-NET 包含 4 个闸门，每个闸门可以获取其特征值，定义如下：

```

////////////////////////////////////
// 闸门判伤结果

typedef struct _ST_GATE_RES
{
    U16     m_xPeak;        // 峰值x坐标
    U16     m_yPeak;        // 峰值y坐标
    U16     m_tPeak;        // 峰值T坐标
    U8      m_fPeak;        // 峰值探伤标志

    U16     m_xpEdge;       // 前沿x坐标
    U16     m_tpEdge;       // 前沿T坐标
}

```

```

    U16      m_xnEdge;          // 后沿x坐标
    U16      m_tnEdge;          // 后沿t坐标
} ST_GATE_RES;

```

4.4.3 累积缓冲区

这是单次累积的数据结构。

```

////////////////////////////////////
// 累积缓冲区

typedef struct _ST_ACC_BUF
{
    U32      m_nSmplDepth;          // 采样深度

    U8      m_bSmplWaveEnb;          // 采样波形使能
    U8      m_bFftWaveEnb;          // FFT波形使能
    U8      m_bCentFreqEnb;          // 中心频率使能

    U32      m_anEncCntr[TOT_ENC_NUM];          // 编码器计数器
    U16      m_anSmplWave[MAX_SMPL_DEPTH];          // 采样波形
    ST_GATE_RES m_astGateRes[TOT_GATE_NUM];          // 闸门特征值
    U32      m_anFftWave[FFT_OUT_DWS];          // FFT波形

    //////////////////////////////////
    // 中心频率

    U32      m_nLeftFreqIndex;          // 左-6dB频率点索引
    U32      m_nMaxFreqIndex;          // 最大频率点索引
    U32      m_nRightFreqIndex;          // 右-6dB频率点索引

    //////////////////////////////////
    // 闸门跟踪

    U8      m_bGateNeedUpd;          // 闸门需要更新
    U32      m_anGateStart[TOT_GATE_NUM - 1];          // 跟踪ABC闸门起点
} ST_ACC_BUF;

```

4.4.4 采集缓冲区

这是累积之后的数据结构。

```

////////////////////////////////////

```

```
// 采集缓冲区
```

```
typedef struct _ST_DAQ_BUF
{
    U32 m_nTotSmplSize;        // 总采样字节数(含附加数据)
    U8 m_nAccTimes;            // 累积次数

    U8 m_nDigIn;                // 数字输入
    F32 m_fCpuTemp;            // CPU温度
    F32 m_fPcbTemp;            // PCB温度
    U32 m_nSmplOrder;          // 采样序号

    // 多次累计数据
    ST_ACC_BUF m_stAccData[MAX_ACC_TIMES][MAX_HARD_NUM][MAX_SOFT_NUM];
} ST_DAQ_BUF;
```

4.5 数据处理线程

存在一个数据捕获线程，接收从 ZXUT-NET 发送过来的原始网络数据。并安装探伤数据结构进行数据解析，之后发送采集完成的事件，通知用户数据处理线程。用户等待采集事件的发生，并调用数据处理线程函数。

```
void CDataProcThrd::Run(LPVOID lpArg)
{
    if (m_hEventAckStopThrd)
    {
        SetPriority(THREAD_PRIORITY_ABOVE_NORMAL);
        while (TRUE)
        {
            if (!m_nTermThrdFlg)
            {
                // 获取采集事件句柄
                HANDLE hDaqEvt = m_pApp->m_pstExpInfo->m_ahDaqEvt[m_nThrdNum];
                if (WaitForSingleObject(hDaqEvt, 10) == WAIT_OBJECT_0)
                {
                    // 调用数据处理线程函数
                    m_pApp->ThrdDataProcHandler(m_nThrdNum);
                }
            }
            else
            {
                break;
            }
        }
    }
}
```

```
        SetEvent (m_hEventPauseThrd);  
    }  
  
    SetEvent (m_hEventAckStopThrd);  
}  
  
CBaseThrd::Run (lpArg);  
}
```

第 5 章 演示软件的使用说明

演示软件的主界面由下面几个界面元素组成，如图 4-1 所示：

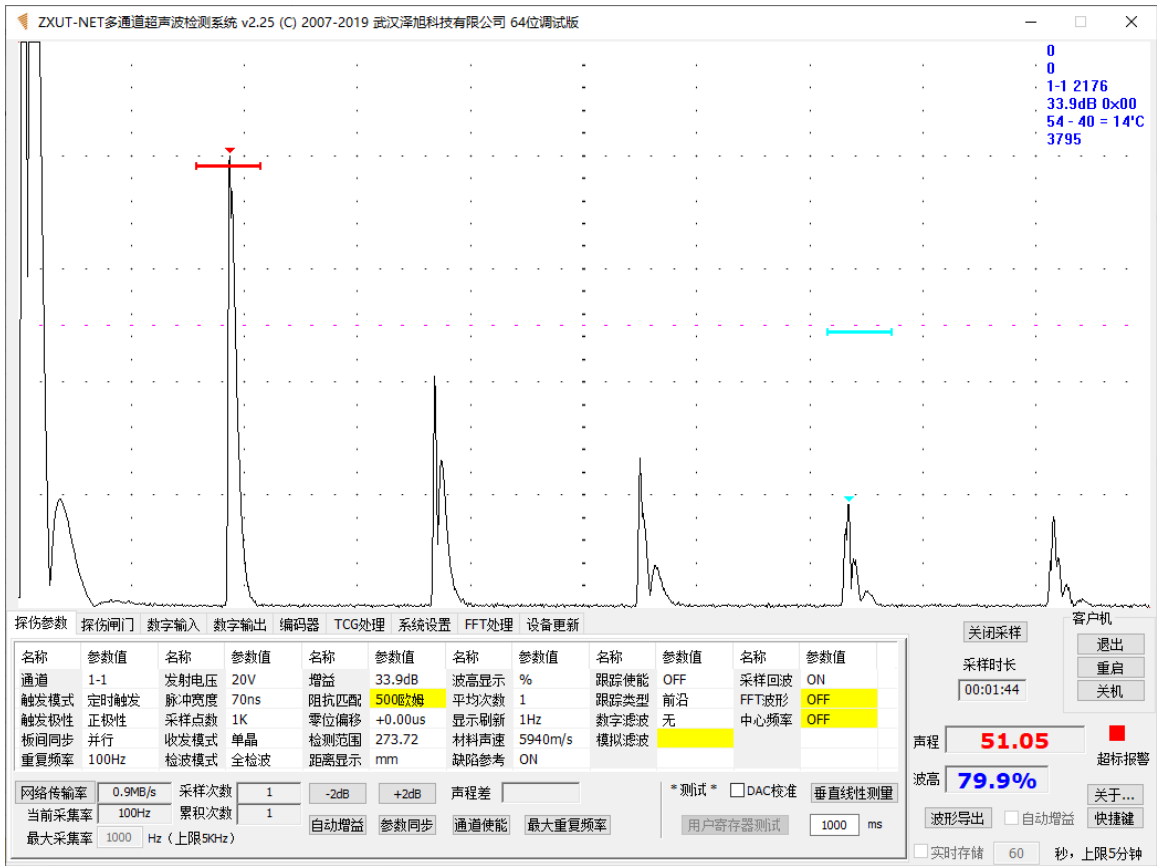


图 5-1：主界面元素

5.1 回波显示区

回波显示区显示当前通道对应的采样回波波形以及闸门位置。任何探伤参数的改变对超声波接收和发射所造成的影响，可以直观地从回波显示区看到。

回波显示区除了采样回波和闸门之外，还在右上角显示下列信息：

67328
67328
1-1 2176
90.0dB 0x00
42 - 26 = 16'C
5807

图 5-2：附加信息

- 67328 - 第一个编码器计数值；
- 67328 - 第二个编码器计数值；

- 1-1 2176 - 1#硬通道, 1#软通道, 通道板的采集量 (双字数);
- 90.0dB 0x00 - 当前增益 90.0dB, 数字输入为 0x00;
- 42 - 26 = 16'C - 核心板温度 42 度, 通道板温度 26 度, 温差 26 度;
- 5807 - 采样序号 (从 1 开始)。

5.2 右侧主面板

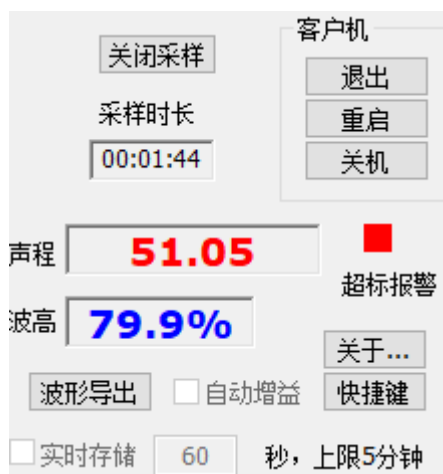


图 5-3: 右侧主面板

5.2.1 开关采样

开启或关闭采样按钮。通过此按钮对 ZXUT 设备的采样状态进行控制。

5.2.2 客户机相关

选择所有客户机“退出”、“重启”和“关机”, 之后演示软件会自动退出。仅用于 ZXUT-NET 探伤设备。

5.2.3 A 闸门特征值

包括缺陷声程和缺陷高度。红色闸门即为缺陷闸门。可以在探伤参数的最下侧来调节闸门的起点、宽度和高度。缺陷闸门内的峰值信息在下图显示: **声程**: 单位 mm; **波高**: 单位 %。

5.2.4 自动增益

当处于自动增益状态时, 会勾选该复选框。

5.2.5 波形导出

以 10 进制 (文件名为 wave_dec.txt) 和 16 进制 (文件名为 wave_hex.txt) 形式导出波形数据到文件。

5.2.6 快捷键

显示演示软件提供的快捷键列表。

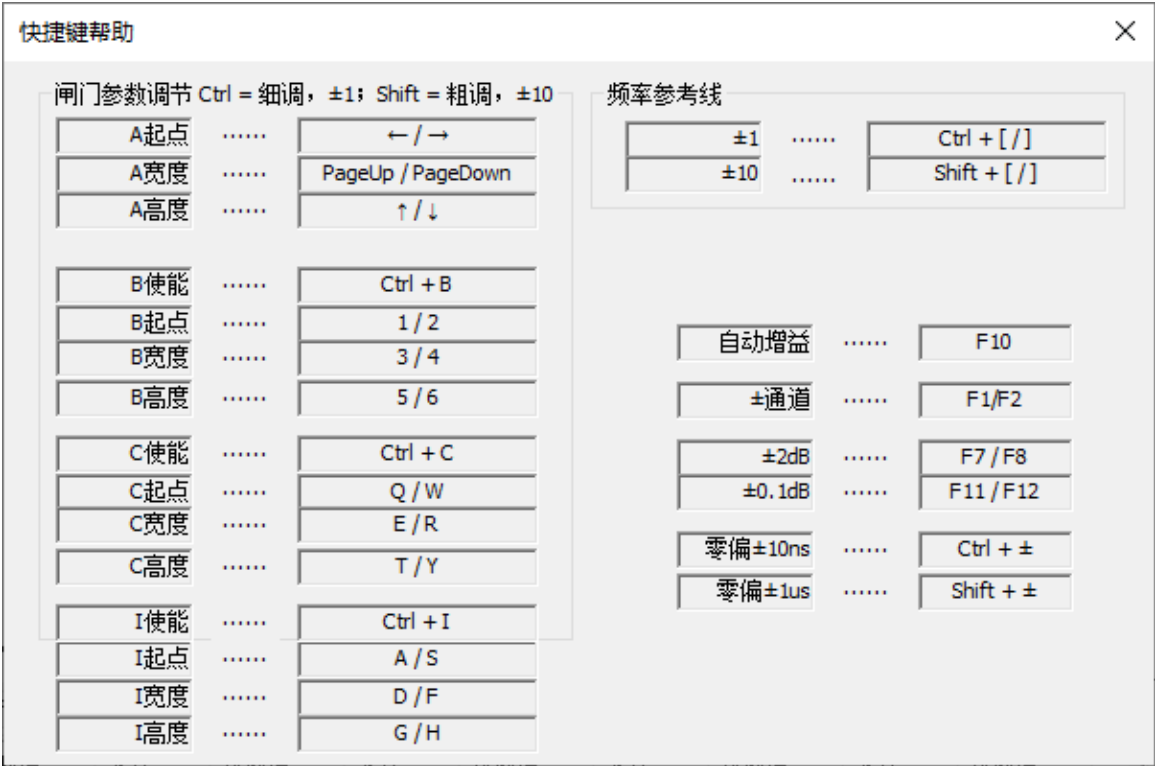


图 5-3：快捷键列表

5.2.7 关于本软件

单击“关于...”按钮，弹出如下对话框：



图 5-4：关于本软件对话框

5.3 调节参数选项卡

5.3.1 探伤参数选项卡



图 5-5：探伤参数选项卡

5.3.1.1 解析系统时序结果



图 5-6：时序解析结果

点击“网络传输率”，可获知当前的网络传输速率。

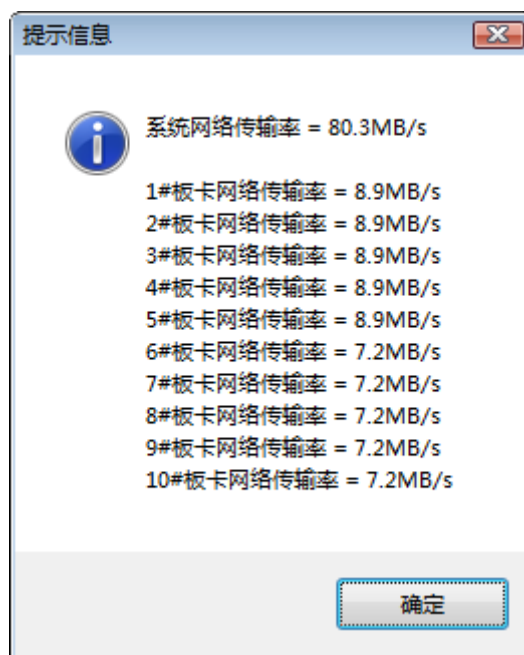


图 5-7：网络数据传输率

另外，还可在此处输入最大采集率（上限 5kHz）。

5.3.1.2 增益粗调

以 $\pm 2dB$ 的步进调节增益。探伤参数调节区中的增益调节，仅能以 0.1dB 为调节步进。有时为了便于测量垂直线性，需要以 2dB 为调节步进。这个 2dB 调节增益按钮就是为这个目的而设置的。

5.3.1.3 自动增益

软件自动调整增益，使得闸门内的波高为 80%。自动增益开始后，右侧主面板的“自动增益”复选框会自动勾选，自动增益完成后再清除勾选。

5.3.1.4 参数同步

使得设备内的所有通道的探伤参数与当前通道相同。

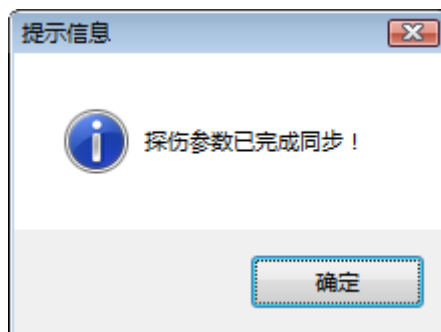


图 5-8：探伤参数同步完成提示

5.3.1.5 探伤参数的调节

此处列举出所有的可以调节的探伤参数。软件开发人员可以调节任何 MUA 设备支持的

探伤参数。这些探伤参数包括：发射脉宽、采样深度、分频比、收发模式、检波模式、零位偏移、数字抑制、阻抗匹配、dB 码以及闸门位置调节。

名称	参数值	名称	参数值	名称	参数值	名称	参数值	名称	参数值	名称	参数值
通道	1-1	发射电压	20V	增益	33.9dB	波高显示	%	跟踪使能	OFF	采样回波	ON
触发模式	定时触发	脉冲宽度	70ns	阻抗匹配	500欧姆	平均次数	1	跟踪类型	前沿	FFT波形	OFF
触发极性	正极性	采样点数	1K	零位偏移	+0.00us	显示刷新	1Hz	数字滤波	无	中心频率	OFF
板间同步	并行	收发模式	单晶	检测范围	273.72	材料声速	5940m/s	模拟滤波			
重复频率	100Hz	检波模式	全检波	距离显示	mm	缺陷参考	ON				

图 5-9：探伤参数调节

5.3.1.6 声程差

若探伤闸门 AB 均开启，会显示两者之间的声程差。

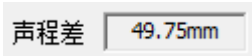


图 5-10：声程差

5.3.1.7 通道使能

点击“通道使能”按钮，可对通道使能进行设置。

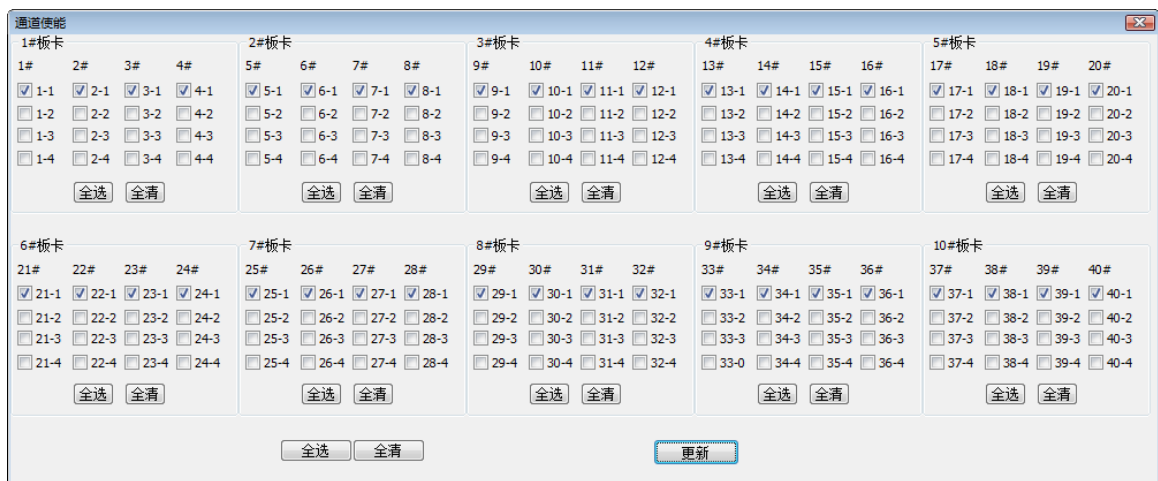


图 5-11：设置通道使能

5.3.2 探伤闸门选项卡

探伤参数	探伤闸门	数字输入	数字输出	编码器	TCG处理	系统设置	FFT处理	通道使能
闸门A	参数值	闸门B	参数值	闸门C	参数值	闸门D	参数值	
闸门起点	178.2	闸门起点	193.1	闸门起点	207.9	闸门起点	222.8	
闸门宽度	17.8	闸门宽度	17.8	闸门宽度	17.8	闸门宽度	17.8	
闸门高度	80.0%	闸门高度	70.0%	闸门高度	60.0%	闸门高度	50.0%	
闸门极性	波峰	闸门极性	波峰	闸门极性	波峰	闸门极性	波峰	
缺陷类型	峰值	缺陷类型	峰值	缺陷类型	峰值	缺陷类型	峰值	
闸门声程	188.09mm	闸门声程		闸门声程		闸门声程		
闸门波高	19.0%	闸门波高		闸门波高		闸门波高		
闸门使能	ON	闸门使能	OFF	闸门使能	OFF	闸门使能	OFF	

图 5-12：探伤闸门选项卡

可对 4 个闸门起点、宽度、高度和极性进行调节。

5.3.3 数字输入选项卡

探伤参数探伤闸门数字输入数字输出编码器TCG处理系统设置FFT处理通道使能

	使能	模式	极性	宽度	延迟		时基
D0	☒	电平	正极性			清零	0.1ms
D1	☒	电平	正极性			清零	
D2	☒	电平	正极性			清零	自动测试全高全低
D3	☒	电平	正极性			清零	DI测试
D4	☒	电平	正极性			清零	☐ 测试使能通道0
D5	☒	电平	正极性			清零	测试频率(20Hz ~ 400Hz)
D6	☒	电平	正极性			清零	400 Hz 宽度 1000 us
D7	☒	电平	正极性			清零	

图 5-13：数字输入选项卡

可对 8 个数字输入（DI）的使能、模式、极性、宽度及延迟进行设置。

5.3.4 数字输出选项卡

探伤参数探伤闸门数字输入数字输出编码器TCG处理系统设置FFT处理通道使能

	使能	模式	极性	宽度	延迟	输出	时基
D0	☒	电平	正极性			触发	0.1ms
D1	☒	电平	正极性			触发	
D2	☒	电平	正极性			触发	自动测试
D3	☒	电平	正极性			触发	
D4	☒	电平	正极性			触发	
D5	☒	电平	正极性			触发	
D6	☒	电平	正极性			触发	全高
D7	☒	电平	正极性			触发	全低

图 5-14：数字输出选项卡

可对 8 个数字输出（DO）的使能、模式、极性、宽度和延迟进行设置。

5.3.5 编码器选项卡

探伤参数探伤闸门数字输入数字输出编码器TCG处理系统设置FFT处理通道使能

使能	正转	扫描	模式	
0# ☒	83859	复位 ☒	☒	四脉冲
1# ☒	83859	复位 ☒	☐	四脉冲
2# ☒	83859	复位 ☒	☐	四脉冲
3# ☒	83859	复位 ☒	☐	四脉冲

☐测试模式
测试频率(100Hz - 500KHz)
100000 Hz
实际频率 100.0KHz
扫描间隔(10-1000) 50
编码器滤波 600KHz
编码器信号<100KHz

图 5-15：编码器选项卡

可对 4 个编码器的使能、极性、模式进行设置。还可以对编码器进行测试。

5.3.6 距离补偿 (TCG) 选项卡

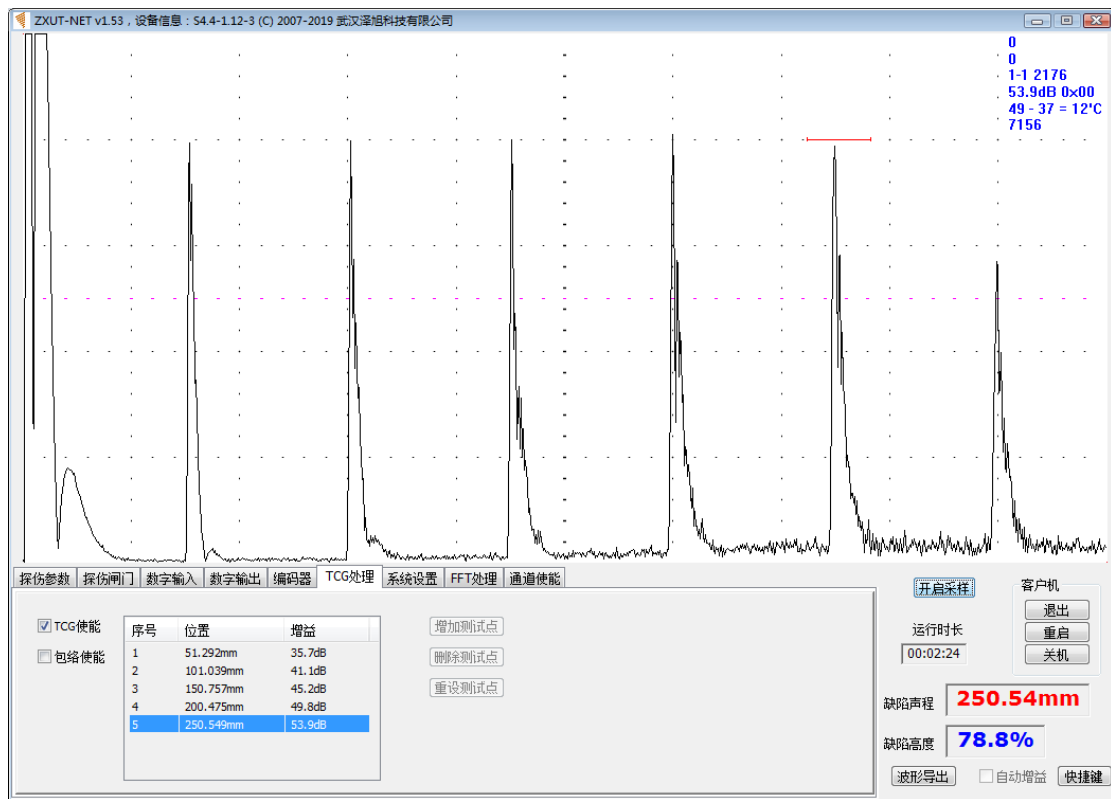


图 5-16: TCG 选项卡

可以增加、删除、复位测试点，以及包络及 TCG 使能设置。

在上图中，一共设定了 5 个测试点。在列表中，分别显示了每个测试点的位置（以距离或时间为单位，取决于在“探伤参数”选项卡中的设置）。当单击“TCG 使能”，开启 TCG 功能后，回波显示如上。

5.3.7 系统设置选项卡



图 5-17: 系统设置选项卡

5.3.7.1 IP 地址设置

IP地址设置

10

0

0

30

客户机首地址

端口

100

8020

服务器地址

备注:

1 客户机和服务器必须在同一个网段;
2 假定客户机首地址为30, 则4块板卡的地址分别为30, 31, 32和33; 服务器地址就不能与上述地址重复。

缺省值

修改

*

图 5-18： IP 地址设置

可以设置客户机首 IP 地址，服务器 IP 地址以及网络端口。

5.3.7.2 板卡当前温度

板卡当前温度 'C										
	1#	2#	3#	4#	5#	6#	7#	8#	9#	10#
核心板	59.6	61.2	60.4	59.4	60.6	59.2	58.3	57.6	57.0	54.6
通道板	45.9	46.3	45.9	45.4	44.8	43.5	43.4	44.1	41.9	40.8
温差	13.7	14.9	14.5	14.0	15.9	15.7	14.9	13.6	15.2	13.8

图 5-19： 板卡当前温度

显示当前核心板和通道板的温度，以及之间的温度差。

5.3.7.3 当前通道板的序号

电路板编号(10位数字：年、月、版本号各2位，序号为4位)

核心板

1806100006

读

写

通道板

1920300041

读

写

高压模块

1812250158

读

写

电源模块

1902110033

读

写

图 5-20： 通道板的序号

5.3.7.4 系统板卡信息

这些信息包括 IP 地址、MAC 地址、各种序号。

通道板信息					
IP地址	MAC地址	核心板	通道板	高压模块	电源模块
10.0.0.30	40:17:BF:AC:00:06	1806100006	1920300041	1812250158	1902110033
10.0.0.31	40:17:BF:AC:00:07	1806100007	1920300042	1812250159	1902110034
10.0.0.32	40:17:BF:AC:00:08	1806100008	1920300043	1812250160	1902110035
10.0.0.33	40:17:BF:AC:00:0B	1806100011	1902300045	1812250162	1902110037
10.0.0.34	40:17:BF:AC:00:0C	1806100012	1902300046	1812250163	1902110038
10.0.0.35	40:17:BF:AC:00:0E	1806100014	1902300048	1812250165	1902110040
10.0.0.36	40:17:BF:AC:00:0F	1806100015	1902300049	1812250166	1902110041
10.0.0.37	40:17:BF:AC:00:26	1806100038	1812300033	1811220153	1806110025
10.0.0.38	40:17:BF:AC:00:04	1806100004	1902300039	1812250156	1902110031
10.0.0.39	40:17:BF:AC:00:17	1806100023	1812300034	1811220154	1806110026

图 5-21：系统板卡信息

5.3.8 FFT 处理选项卡



图 5-22：FFT 处理选项卡

可硬件实现 FFT 功能。

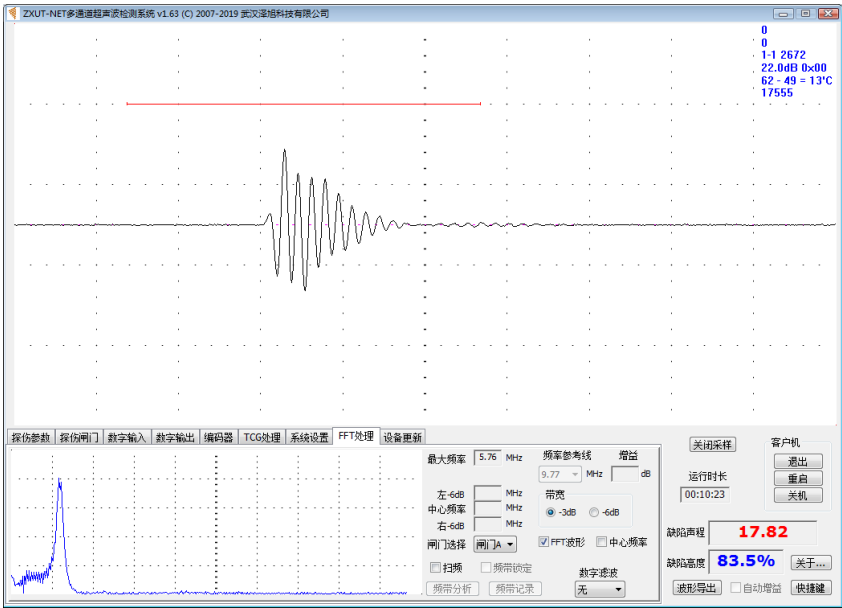


图 5-23：FFT 波形

5.4 实时存储（RtMem）

在某些应用场合，用户需要把每次采样波形全部存储在硬盘上。若使用普通的访问硬盘的方式，且重复频率又非常的高，就不能保证实时存储。

为此，必须使用文件内存映射的方式，将硬盘文件映射到内存中。先将采样数据缓存于内存中，必要时再将内存数据搬移到硬盘。

要实现实时存储，推荐确保如下三个条件：

- （1）使用 64 位操作系统；
- （2）安装较大的内存，比如 8GB 或 16GB；
- （3）至少使用 7200 转的高速硬盘，或者直接使用固态硬盘（SSD）。

具体进行实时存储的步骤如下：

5.4.1 关闭采样，设置实时存储参数

输入存储时间（秒数）60，上限位 300 秒即 5 分钟。勾选“实时存储”，将弹出如下存储信息对话框：



图 5-24：实时存储信息对话框

- 通道：1-1；
- 时长：60 秒，先前指定；
- 次数：60,000，60 秒 × 100 次/秒 = 60,000 次；
- 文件名称：RtMem.mm，可在 API 函数 ZXUT_InitRtMem 中指定；
- 文件长度：12,288,000 = 60,000 × 1 × 1024 × 2。对于 12 位 ADC，1 次采样需要占用 2 个字节；对于 8 位，仅占用 1 个字节；
- 状态：正常，分配内存及硬盘文件正常。

5.4.2 开启采样，等待实时存储结束

开启采样，演示软件将自动进行实时存储。用户可以通过查看“采样时长”，来确定实时存储是否已经完成。此处设置为 60 秒，为了保险起见，延迟 10 秒钟只有，停止采样。演示软件会将内存中的采样数据，全部存储于硬盘。

5.4.3 实时存储数据的确认

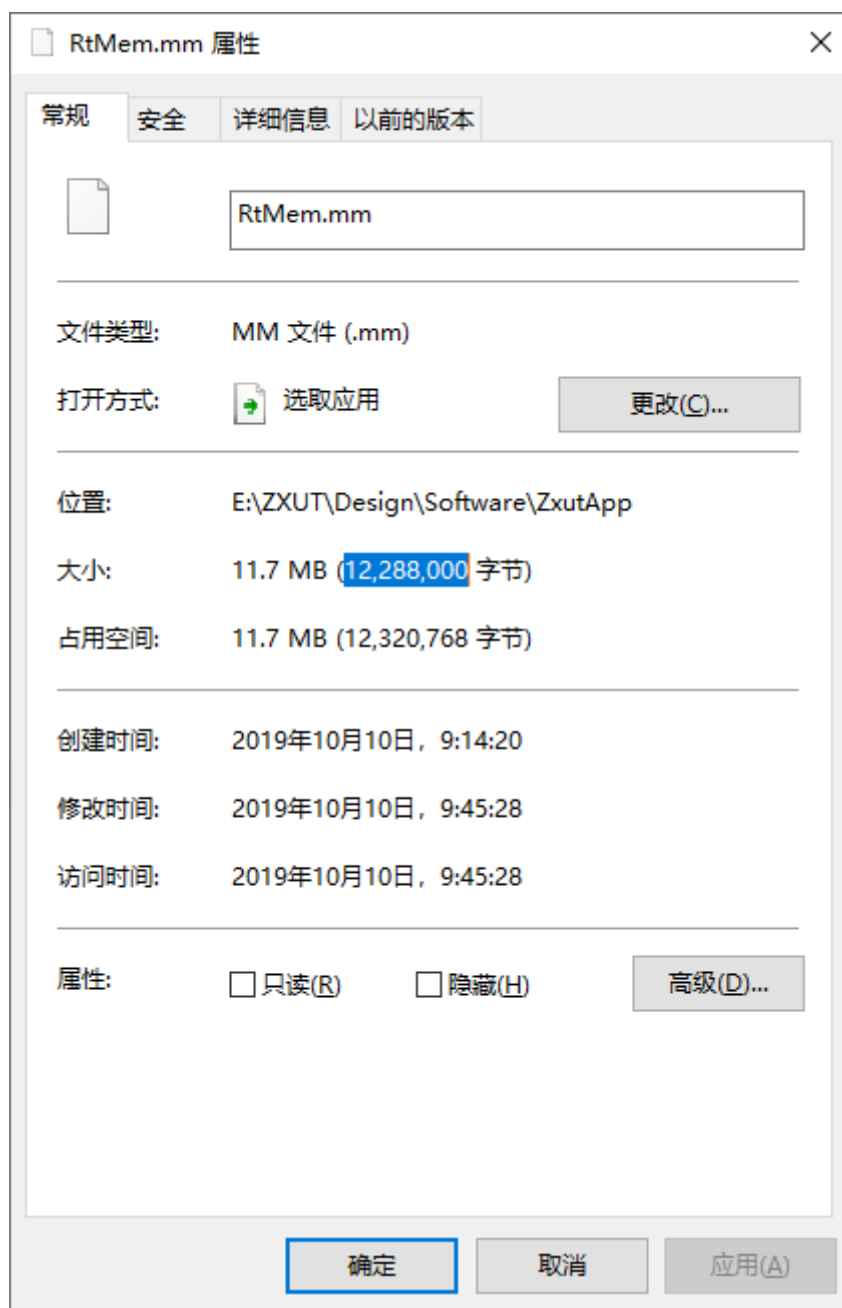


图 5-25：实时存储硬盘文件 RtMem.mm 的文件属性

与该文件一起的还有一个 RtMem.txt 文件，文本形式存储最后 10 次的采样波形数据。



图 5-26: 对应的最后 10 次采样的文本文件

将上述数据导入到 EXCEL 表格中，并以折线形式显示出来，如下图。

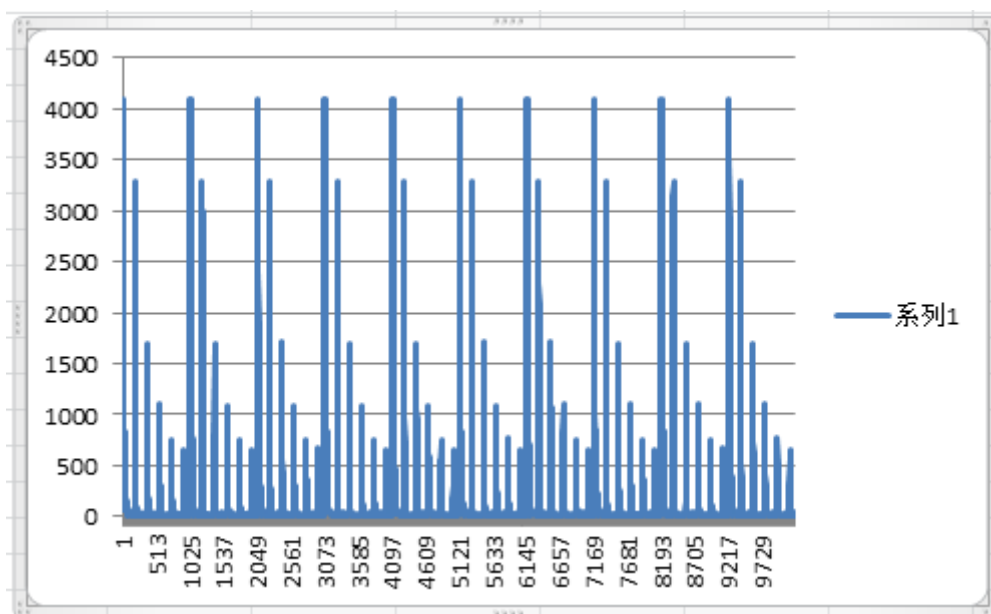


图 5-27: 导入到 EXCEL 表格中的采样数据

